ADITYA UNIVERSITY

SCHOOL OF ENGINEERING

Department of Computer Science and Engineering

CERTIFICATE

This is to certify that the Cornerstone project work entitled *"***CARA – E-COMMERCE WEB APPLICATION***"* is submitted by

A.ASHOK	**(25B21CS019)**
S.SRI ANJAN KUMAR	**(25B21CS017)**
A. NOOKARAJU	**(25B21CS020)**
J.PRANADEEP KUMAR	**(24B11CS161)**

in partial fulfillment of the requirements for the award of the B.Tech. degree in Computer Science and Engineering for the academic year 2025-2026.

Project Guide
Mr. Bhanu Prakash,M.Tech.,(Ph.D)
Assistant Professor,
Department of CSE

Head of the Department
Dr. T.Sudha Rani, M.Tech.,Ph.D
Assistant Professor,
Department of CSE

DECLARATION

We hereby declare that the Cornerstone project entitled "**CARA – E-COMMERCE WEB APPLICATION**" is original and is submitted to **ADITYA UNIVERSITY**, Surampalem, in partial fulfillment of the requirements for the award of the **B.Tech. degree** in **Computer Science and Engineering**. We further declare that this project work has not been submitted, either in full or in part, for the award of any degree in this or any other institution. We also confirm that this work complies with the Institutional Academic Integrity and AI Usage Policy

Place: Surampalem

Date:

By

A.ASHOK	**(25B21CS019)**
S.SRI ANJAN KUMAR	**(25B21CS017)**
A. NOOKARAJU	**(25B21CS020)**
J.PRANADEEP KUMAR	**(24B11CS161)**

ACKNOWLEDGEMENT

It is with immense pleasure that we express our sincere gratitude to our Cornerstone Project Guide, **Mr. Bhanu Prakash, Assistant Professor**, who has guided us throughout the project. His valuable support, encouragement, and guidance have greatly contributed to the successful completion of this work.

We would like to thank our Deputy Heads of the Department, **Dr. Hari Jyothula Associate Professor and Dr.K.Chandra Shekar, Associate professer**, Department of CSE, for their support and valuable suggestions.

Our deepest thanks to our Head of the Department ,**Dr.T.Sudha Rani, Assistant Professor,** for his inspiration and for providing all the necessary facilities and resources required for this project.

We also extend our sincere thanks to **Dr. M.V. Rajesh, Associate Dean, School of Engineering** for their encouragement and support during the course of our project.

We would like to express our sincere gratitude to **Dr.G.Suresh,Registrar, Dr.A Ramesh, ProVice-Chancellor(Engineering&Sciences), Dr.S.Rama Sree,ProVice-Chancellor (Academics), Dr.M.B.Srinivas, Vice-Chancellor, Dr. M.Sreenivasa Reddy, Deputy Pro Chancellor and Management, Aditya University** for providing excellent infrastructure and state-of-the-art laboratories.

Finally, we would like to thank the **faculty members, lab technicians, non-teaching staff, and our friends** who have directly or indirectly supported us in completing this Cornerstone project successfully.

TABLE OF CONTENTS

Description	Page No.
Abstract	i
List of Figures	ii
List of Tables	iii
Acronyms	iv
1. INTRODUCTION	1
1.1 Brief Information about the Project	1
1.2 Motivation and Contribution of the Project	1
1.3 Objectives of the Project	1
1.4 Scope of the Project	2
2. SYSTEM ANALYSIS	3
2.1 Functional Requirements	3
2.2 Non-Functional Requirements	3
2.3 Requirements Specification	4
3. TECHNOLOGY DESCRIPTION	5
3.1 Programming Languages	5
3.2 Frameworks and Libraries	5
3.3 Database Technology	6
3.4 Tools and IDEs	6
4. DATABASE DESIGN	7
4.1 Database Tables (Collections)	7
4.2 Collection Structure	7
4.3 Keys and Constraints	8
4.4 Collection Relationships	8
5. SYSTEM DESIGN	9
5.1 System Architecture	9
5.2 Module Descriptions	10
6. SYSTEM IMPLEMENTATION	11
7. CONCLUSION	15
8. FUTURE ENHANCEMENTS	16
9. REFERENCES	17
10. Appendex	18

ABSTRACT

Cara is a full-stack e-commerce web application developed using the MERN Stack (MongoDB, Express.js, React.js, Node.js). The platform enables users To browse products, add items to cart, and place orders efficiently.
The system provides secure authentication, dynamic product management, and A responsive UI. It demonstrates real-world implementation of modern web Technologies for online shopping solutions.

Additionally, the application ensures high performance, scalability, and Reliability. It incorporates efficient database handling, modular architecture, And user-friendly design principles. The system supports seamless interaction Between frontend and backend, enabling smooth data flow and real-time Updates.

LIST OF FIGURES

Figure No.	Figure Name	Page No.
6.1	Login Page	11
6.2	Home Page	12
6.3	Features	12
6.4	All Products Page	13
6.5	Product Detail Page	13
6.6	Search items	14
6.7	Check-out items	
6,8	Admin portal	

LIST OF TABLES

Table No.	Table Name	Page No.
4.2.1	Users Collection Structure	7
4.2.2	Books Collection Structure	7
4.2.3	CartItems Collection Structure	8
4.2.4	Otp Collection Structure	9

ACRONYMS

Acronym	Full Form
MERN	MongoDB, Express.js, React, Node.js
JWT	JSON Web Token
API	Application Programming Interface
REST	Representational State Transfer
CRUD	Create, Read, Update, Delete
HTTP	Hypertext Transfer Protocol
HTTPS	Hypertext Transfer Protocol Secure
UI	User Interface
UX	User Experience
JSON	JavaScript Object Notation
MVC	Model-View-Controller
DB	Database
CSS	Cascading Style Sheets
HTML	Hypertext Markup Language
DOM	Document Object Model
NPM	Node Package Manager
ODM	Object Data Modeling
SPA	Single Page Application
CORS	Cross-Origin Resource Sharing

1. INTRODUCTION

1.1 Brief Information about the Project

Cara is a full-stack e-commerce web application designed to provide users with a seamless and Efficient online shopping experience. The application is developed using the MERN stack, which Includes MongoDB for database management, Express.js and Node.js for backend development, And React.js for building a dynamic and responsive user interface.
Overall, Cara demonstrates the practical implementation of modern web technologies to build a Scalable and secure e-commerce platform, making it suitable for real-world applications.

1.2 Motivation and Contribution of the Project

The rapid growth of digital technology and internet usage has significantly transformed the way people shop. Traditional shopping methods are time-consuming and limited by physical location, while many existing online platforms are often complex, slow, or not user-friendly. This created the need for a simple, efficient, and scalable e-commerce solution.

Additionally, the project was motivated by the need to gain practical experience in modern web technologies such as React, Node.js, and MongoDB, which are widely used in the industry.

1.3 Objectives of the Project

- To design and develop a responsive e-commerce website using React.js
- To implement a secure user authentication and authorization systemBuild
- To enable users to browse, search, and filter products بسهولة and efficiently
- To provide a shopping cart system for managing selected products
- To implement a smooth and secure checkout process

1.4 Scope of the Project

- User Management: User registration and login mechanisms protected with secure JWT authentication.
- Shopping Cart: Add products to cart Remove or update product quantities

  View total price and selected items

- User Features: Cart management functionality allowing users to add, remove, and checkout items.
- Admin Features: A dedicated admin panel for complete and menu lifecycle management including CRUD operations.
- Accessibility: A fully responsive web interface accessible across desktop, tablet, and mobile devices.
- Data Storage: Utilizing MongoDB database , document-based, and scalable storage.

2. SYSTEM ANALYSIS

2.1 Functional Requirements

Functional requirements outline the specific behaviors and functions the system must support. For TasteFlow, these include:

- User Registration and Login: The system must allow users to create an account and authenticate.
- JWT-based Authentication: The system must manage sessions securely via tokens.
- Browse Products and Menus: Users must be able to view all available Products and Their menu items.
- Add Items to Cart: Users must be able to add selected Products items to their shopping cart with quantity controls.
- Order Checkout: The system must permit users to review their cart and proceed to checkout with an order summary.
- Admin Management: Administrators must be able to add, edit, or delete products and menu records.
- Role-Based Access Control: The system must restrict certain API routes based on user roles (Admin vs User).
- Secure Password Storage: The system must hash user passwords using bcrypt before saving them to the database.

2.2 Non-Functional Requirements

- Performance: The application should exhibit fast response times, ensuring API queries resolve under 500ms.
- Security: All sensitive endpoints must require valid JWT headers, and passwords must never be stored in plain text.
- Scalability: Utilizing MongoDB allows the database to support horizontal scaling as the user base and restaurant catalog grow.
- Usability: The React UI must be intuitive, modern, and highly responsive across various screen sizes.
- Reliability: The backend server must gracefully handle errors to target a 99.9% uptime scenario.

2.3 Requirements Specification

2.3.1 Hardware Requirements

- Processor: 1 GHz or faster processor
- RAM: 4 GB minimum (8 GB recommended for development)
- Disk Space: 10 GB of free space
- Network: Stable Internet connection for cloud database interactions

2.3.2 Software Requirements

- Environment: Node.js v18.x or higher
- Database: MongoDB Atlas (Cloud) or local MongoDB instance
- Browser: Google Chrome, Mozilla Firefox, or Microsoft Edge

3. TECHNOLOGY DESCRIPTION

3.1 Programming Languages

3.1.1 JavaScript

JavaScript is a versatile, high-level programming language that conforms to the ECMAScript specification. In TasteFlow, JavaScript serves as the foundational language across the entire stack, powering the interactive React components on the frontend as well as the server-side logic and database models on the backend via Node.js.

3.2 Frameworks and Libraries

3.2.1 React

React is a prominent open-source JavaScript library for building user interfaces, specifically single-page applications. It provides enhanced concurrent rendering features and hooks that allow for efficient DOM manipulation, providing a seamless browsing experience for food delivery users.

3.2.2 Vite

Vite is a modern frontend build tool that significantly improves the development experience. By utilizing native ES modules, Vite offers exceptionally fast hot module replacement (HMR) and optimized build processes, serving as the build environment for the React application.

3.2.3 Express.js

Express.js is a minimal and flexible Node.js web application framework providing a robust set of features for web and mobile applications. It handles routing, middleware integration, and HTTP request processing, forming the backbone of the system's RESTful API.

3.2.4 bcryptjs

bcryptjs is a cryptographic library used to securely hash passwords. By applying a mathematical salt and hashing algorithm, bcrypt ensures that user passwords cannot be reverse-engineered, adding a critical layer of security to the database.

3.2.5 JSON Web Tokens (JWT)

JWT is an open standard that defines a compact and self-contained way for securely

transmitting information between parties as a JSON object. It is used in this system for maintaining stateless authentication sessions and enforcing role-based access control.

3.3 Database Technology

3.3.1 MongoDB

MongoDB is a source-available, cross-platform document-oriented database program. Classified as a NoSQL database, MongoDB uses JSON-like documents with optional schemas. It provides the flexibility required to store dynamic restaurant metadata, menu items, user profiles, and order information efficiently.

3.3.2 Mongoose ODM

Mongoose is an Object Data Modeling (ODM) library for MongoDB and Node.js. It manages relationships between data, provides schema validation, and translates between objects in code and the representation of those objects in MongoDB.

1.1 Tools and IDEs

1.1.1 Visual Studio Code

VS Code is a lightweight but powerful source code editor providing deep integrations for JavaScript, Node.js, and Git. It serves as the primary development environment for writing both the frontend and backend codebases.

1.1.2 Postman / MongoDB Compass

Postman is utilized for designing, testing, and documenting the REST API endpoints during development. MongoDB Compass provides a graphical user interface for visualizing database schemas, querying data, and managing collections directly.

4. DATABASE DESIGN

4.1 Database Tables (Collections)

In a MongoDB environment, traditional relational tables are replaced by collections. The TasteFlow system relies primarily on five collections: Users (managing authentication and roles), Restaurants (storing restaurant metadata), MenuItems (storing food items), Orders (tracking user orders), and CartItems (tracking user cart selections).

4.2 Collection Structure

Table 4.2.1: Users Collection Structure

Column Name (Field)	Data Type	Constraint / Description
_id	ObjectId	Primary Key, Auto-generated
name	String	Required
email	String	Required, Unique constraint
password	String	Required, Hashed format
role	String	Enum: 'admin', 'user'. Default: 'user'
createdAt	Date	Default: Date.now()

Table 4.2.2: Restaurants Collection Structure

Column Name (Field)	Data Type	Constraint / Description
_id	ObjectId	Primary Key, Auto-generated
name	String	Required
description	String	Optional textual description
address	String	Required
rating	Number	Default: 0
imageUrl	String	Path to the restaurant cover image
createdAt	Date	Default: Date.now()

1.2 Keys and Constraints

Table 4.2.3: MenuItems Collection Structure

Column Name (Field)	Data Type	Constraint / Description
_id	ObjectId	Primary Key, Auto-generated
restaurantId	ObjectId	Foreign Key referencing Restaurants
name	String	Required
description	String	Optional

price	Number	Required

category	String	Required
imageUrl	String	Path to the food item image

Table 4.2.4: Orders Collection Structure

Column Name (Field)	Data Type	Constraint / Description
_id	ObjectId	Primary Key, Auto-generated
userId	ObjectId	Foreign Key referencing Users
items	Array	Array of { menuItemId, quantity, price }
totalAmount	Number	Required
status	String	Enum: 'pending', 'confirmed', 'delivered'
createdAt	Date	Default: Date.now()

Table 4.2.5: CartItems Collection Structure

Column Name (Field)	Data Type	Constraint / Description
_id	ObjectId	Primary Key, Auto-generated
userId	ObjectId	Foreign Key referencing Users
menuItemId	ObjectId	Foreign Key referencing MenuItems
quantity	Number	Default: 1
addedAt	Date	Default: Date.now()

4.3 Collection Relationships

The schema design exhibits a one-to-many relationship between Restaurants and MenuItems, and a one-to-many relationship between Users and CartItems. A specific CartItem document acts as a junction entity, binding an individual User's ObjectId to a specific MenuItem's ObjectId. Orders maintain a reference to the user and embed item details for historical accuracy. This setup allows quick population mapping using Mongoose's .populate() method, dynamically assembling the user's cart and order history interfaces.

5 SYSTEM DESIGN

5.2 System Architecture

TasteFlow is architected based on a modern Three-Tier MVC (Model-View-Controller) pattern adapted for Single Page Applications (SPAs). The presentation tier is developed using React, rendering the UI dynamically on the client's browser. The application tier, governed by Express.js and Node.js, processes business logic, securely handles authentication via middleware, and exposes RESTful API endpoints. The data tier is supported by MongoDB, maintaining all persistent application state.

CLIENT BROWSER (React Frontend)
↓↑ HTTP/REST Requests (JSON)
APPLICATION SERVER (Express.js + Node.js API)
↓↑ Mongoose ODM queries
DATABASE SERVER (MongoDB)

5.3 Module Descriptions

- **User Module:** Handles user profile creation, validation, and session state initialization on the frontend.
- **Login/Authentication Module:** Receives credentials, compares bcrypt hashes, and securely issues and validates JWT tokens across API boundaries.
- **Admin Module:** A restricted section accessible only to users with the 'admin' role, enabling restaurant oversight, analytics viewing, and catalog manipulation.
- **Cart Module:** Manages the temporary queuing of selected food items, syncing local state with the backend CartItems collection.
- **Menu/Navigation Module:** A responsive routing mechanism (using React Router) that dynamically renders navbar links based on authentication status and user roles.
- **Product Management Module:** The core CRUD interface for processing products uploads, modifying metadata, handling cover images, and managing menu items.

4.APPENDIX

A.1 Sample Code – Backend Authentication Route

```
// routes/auth.js
const express = require('express');
const bcrypt  = require('bcryptjs');
const jwt     = require('jsonwebtoken');
const User    = require('../models/User');
const router  = express.Router();

router.post('/login', async (req, res) => {
  try {
    const  { email,  password } = req.body;
    const user = await User.findOne({ email });
    if (!user) return res.status(404).json({ message: 'User not found' });
    const isMatch = await bcrypt.compare(password, user.password);
    if (!isMatch) return res.status(400).json({ message: 'Invalid credentials' });
    const token = jwt.sign(
      { id: user._id, role: user.role },
      process.env.JWT_SECRET,
      { expiresIn: '1d' }
    );
    res.status(200).json({ token, user: { id: user._id, name: user.name, role:
user.role } });
  } catch (error) {
    res.status(500).json({ message: 'Server error' });
   }
});
module.exports = router;
```

A.1 Sample Code – Frontend React Cart Component

```
// components/CartItem.jsx
import React from 'react';

const CartItem = ({ item, onQuantityChange, onRemove }) => {
  return (
    <div className='cart-item border p-4 shadow-sm flex items-center'>
      <img src={item.imageUrl} alt={item.name} className='w-24 h-24 object-cover
rounded'/>
      <div className='ml-4 flex-1'>
        <h3 className='text-lg font-bold'>{item.name}</h3>
        <p className='text-orange-500 font-semibold'>{item.price}.00</p>
        <div className='flex items-center mt-2 gap-2'>
          <button onClick={() => onQuantityChange(item._id, -1)}>-</button>
          <span>{item.quantity}</span>
          <button onClick={() => onQuantityChange(item._id, 1)}>+</button>
          <button onClick={() => onRemove(item._id)}>Remove</button>
        </div>
      </div>
    </div>
  );
};
export default CartItem;
```

A.2 Sample Code – Tailwind.config.js

```
/** @type {import('tailwindcss').Config} */
export default {
  content: [
    "./index.html",
    "./src/**/*.{js,ts,jsx,tsx}",
  ],
  theme: {
    extend: {
      fontFamily: {
        sans: ['Inter', 'sans-serif'],
      },
      colors: {
        brand: {
          50: '#fff1f2',
          100: '#ffe4e6',
          500: '#f43f5e',
          600: '#e11d48',
          900: '#881337',
        }
      },
      animation: {
        'fade-in': 'fadeIn 0.5s ease-out',
        'slide-up': 'slideUp 0.5s ease-out',
      },
      keyframes: {
        fadeIn: {
          '0%': { opacity: '0' },
          '100%': { opacity: '1' },
        },
        slideUp: {
          '0%': { opacity: '0', transform: 'translateY(20px)' },
          '100%': { opacity: '1', transform: 'translateY(0)' },
        }
      }
    },
  },
  plugins: [],
}
```

A.3 Sample Code-Autharisation.js

```
import React, { useState } from 'react';
import { useNavigate, Link } from 'react-router-dom';
import { FaUserCircle, FaEnvelope, FaLock } from 'react-icons/fa';

const LoginPortal = () => {
  const navigate = useNavigate();
  const [email, setEmail] = useState('');
  const [password, setPassword] = useState('');
  const [error, setError] = useState('');

  const handleLogin = async (e) => {
    e.preventDefault();
    setError('');

    try {
      const res = await fetch('http://localhost:5000/api/auth/login', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({ email, password })
      });
      const data = await res.json();

      if (!res.ok) {
        throw new Error(data.message || 'Login failed');
      }

      // Save token and user info
      localStorage.setItem('token', data.token);
      localStorage.setItem('user', JSON.stringify(data.user));

      // Redirect based on role
      if (data.user.role === 'admin') {
        navigate('/admin');
      } else {
        navigate('/');
      }
```

```
    } catch (err) {
      setError(err.message || 'Invalid email or password. Please try admin@admin.com / admin123');
    }
  };

  return (
    <div className="min-h-screen bg-[#fcf9f6] flex items-center justify-center p-4 relative overflow-hidden">
      {/* Subtle light background circles for aesthetic */}
      <div className="absolute top-[-10%] left-[-10%] w-[40%] h-[40%] bg-[#f5e6de] rounded-full blur-[100px] opacity-80"></div>
      <div className="absolute bottom-[-10%] right-[-10%] w-[40%] h-[40%] bg-[#e8f0ed] rounded-full blur-[100px] opacity-80"></div>

      <div className="w-full max-w-md bg-white border border-gray-100 rounded-3xl p-10 shadow-[0_8px_30px_rgb(0,0,0,0.04)] relative z-10 transition-all duration-300">
        <div className="flex justify-center mb-8">
          <FaUserCircle className="text-6xl text-gray-800" />
        </div>

        <h2 className="text-3xl font-bold text-center text-gray-900 mb-2 tracking-tight">Welcome Back</h2>
        <p className="text-center text-gray-500 mb-8 font-light text-sm">Please sign in to access the portal</p>

        <form onSubmit={handleLogin} className="space-y-6">
          {error && (
            <div className="bg-red-50 text-red-600 p-3 rounded-xl text-sm text-center border border-red-100">
              {error}
            </div>
          )}
          <div className="relative">
            <div className="absolute inset-y-0 left-0 pl-3 flex items-center pointer-events-none">
              <FaEnvelope className="text-gray-400" />
            </div>
            <input
              type="email"
              required
```

```
                value={email}
                onChange={(e) => setEmail(e.target.value)}
                className="w-full pl-10 pr-4 py-3 bg-gray-50 border border-gray-200 rounded-xl focus:outline-none focus:ring-2 focus:ring-gray-900 text-gray-900 placeholder-gray-400 transition-all"
                placeholder="Email Address"
            />
        </div>

        <div className="relative">
            <div className="absolute inset-y-0 left-0 pl-3 flex items-center pointer-events-none">
                <FaLock className="text-gray-400" />
            </div>
            <input
                type="password"
                required
                value={password}
                onChange={(e) => setPassword(e.target.value)}
                className="w-full pl-10 pr-4 py-3 bg-gray-50 border border-gray-200 rounded-xl focus:outline-none focus:ring-2 focus:ring-gray-900 text-gray-900 placeholder-gray-400 transition-all"
                placeholder="Password"
            />
        </div>

        <div className="flex items-center justify-between text-sm">
            <label className="flex items-center text-gray-600 cursor-pointer hover:text-gray-900 transition-colors">
                <input type="checkbox" className="mr-2 rounded border-gray-300 bg-white text-gray-900 focus:ring-gray-900" />
                Remember me
            </label>
            <a href="#" className="text-gray-500 hover:text-gray-900 transition-colors font-medium">Forgot Password?</a>
        </div>

        <button
            type="submit"
            className="w-full bg-gray-900 hover:bg-black text-white font-bold py-3.5 px-4 rounded-xl shadow-md transform hover:-translate-y-0.5 transition-all duration-200"
```

```
                >
                    Sign In
                </button>


                <p className="text-center text-gray-500 text-sm mt-6">
                    Don't have an account? <Link to="/signup" className="text-gray-900 font-medium
hover:underline transition-all">Sign up</Link>
                </p>
            </form>
        </div>
    </div>
  );
};
```

A.4 Sample-code MongoDB Connectioons

```
'use strict';

/**
 * Dependencies
 */

const assert = require('assert');
const util = require('util');
const utils = require('./utils');

const debug = util.debuglog('mquery');

/**
 * Query constructor used for building queries.
 *
 * #### Example:
 *
 *     var query = new Query({ name: 'mquery' });
 *     query.setOptions({ collection: moduleCollection })
 *     await query.where('age').gte(21).exec();
 *
 * @param {Object} [criteria] criteria for the query OR the collection instance to use
 * @param {Object} [options]
 * @api public
 */

function Query(criteria, options) {
  if (!(this instanceof Query))
    return new Query(criteria, options);

  const proto = this.constructor.prototype;

  this.op = proto.op || undefined;

  this.options = Object.assign({}, proto.options);

  this._conditions = proto._conditions
    ? utils.clone(proto._conditions)
    : {};

  this._fields = proto._fields
    ? utils.clone(proto._fields)
    : undefined;
```

```
  this._updateDoc = proto._updateDoc
    ? utils.clone(proto._updateDoc)
    : undefined;

  this._path = proto._path || undefined;
  this._distinctDoc = proto._distinctDoc || undefined;
  this._collection = proto._collection || undefined;
  this._traceFunction = proto._traceFunction || undefined;

  if (options) {
    this.setOptions(options);
  }

  if (criteria) {
    this.find(criteria);
  }
}

/**
 * Converts this query to a constructor function with all arguments and options retained.
 *
 * #### Example:
 *
 *     // Create a query that will read documents with a "video" category from
 *     // `aCollection` on the primary node in the replica-set unless it is down,
 *     // in which case we'll read from a secondary node.
 *     var query = mquery({ category: 'video' })
 *     query.setOptions({ collection: aCollection, read: 'primaryPreferred' });
 *
 *     // create a constructor based off these settings
 *     var Video = query.toConstructor();
 *
 *     // Video is now a subclass of mquery() and works the same way but with the
 *     // default query parameters and options set.
 *
 *     // run a query with the previous settings but filter for movies with names
 *     // that start with "Life".
 *     Video().where({ name: /^Life/ }).exec(cb);
 *
 * @return {Query} new Query
 * @api public
 */

Query.prototype.toConstructor = function toConstructor() {
  function CustomQuery(criteria, options) {
    if (!(this instanceof CustomQuery))
      return new CustomQuery(criteria, options);
    Query.call(this, criteria, options);
  }

  utils.inherits(CustomQuery, Query);

  // set inherited defaults
  const p = CustomQuery.prototype;

  p.options = {};
  p.setOptions(this.options);

  p.op = this.op;
  p._conditions = utils.clone(this._conditions);
  p._fields = utils.clone(this._fields);
  p._updateDoc = utils.clone(this._updateDoc);
  p._path = this._path;
  p._distinctDoc = this._distinctDoc;
  p._collection = this._collection;
  p._traceFunction = this._traceFunction;

  return CustomQuery;
};
```

```
/**
 * Sets query options.
 *
 * #### Options:
 *
 * - [tailable](http://www.mongodb.org/display/DOCS/Tailable+Cursors) *
 * - [sort](http://www.mongodb.org/display/DOCS/Advanced+Queries#AdvancedQueries-%7B%7Bsort(\)%7D%7D) *
 * - [limit](http://www.mongodb.org/display/DOCS/Advanced+Queries#AdvancedQueries-%7B%7Blimit%28%29%7D%7D) *
 * - [skip](http://www.mongodb.org/display/DOCS/Advanced+Queries#AdvancedQueries-%7B%7Bskip%28%29%7D%7D) *
 * - [maxTime](http://docs.mongodb.org/manual/reference/operator/meta/maxTimeMS/#op._S_maxTimeMS) *
 * - [batchSize](http://www.mongodb.org/display/DOCS/Advanced+Queries#AdvancedQueries-%7B%7BbatchSize%28%29%7D%7D) *
 * - [comment](http://www.mongodb.org/display/DOCS/Advanced+Queries#AdvancedQueries-%24comment) *
 * - [hint](http://www.mongodb.org/display/DOCS/Advanced+Queries#AdvancedQueries-%24hint) *
 * -  [slaveOk](http://docs.mongodb.org/manual/applications/replication/#read-preference)  *
 * - [safe](http://www.mongodb.org/display/DOCS/getLastError+Command)
 * - collection the collection to query against
 *
 * _* denotes a query helper method is also available_
 * @param {Object} options
 * @api public
 */

Query.prototype.setOptions = function(options) {
  if (!(options && utils.isObject(options)))
    return this;

  // set arbitrary options
  const methods = utils.keys(options);
  let method;

  for (let i = 0; i < methods.length; ++i) {
    method = methods[i];

    // use methods if exist (safer option manipulation)
    if ('function' == typeof this[method]) {
      const args = Array.isArray(options[method])
        ? options[method]
        : [options[method]];
      this[method].apply(this, args);
    } else {
      this.options[method] = options[method];
    }
  }

  return this;
};

/**
 * Sets this Querys collection.
 *
 * @param {Collection} coll
 * @return {Query} this
 */

Query.prototype.collection = function collection(coll) {
  this._collection = new Query.Collection(coll);

  return this;
```

```
};

/**
 * Adds a collation to this op (MongoDB 3.4 and up)
 *
 * #### Example:
 *
 *     query.find().collation({ locale: "en_US", strength: 1 })
 *
 * @param {Object} value
 * @return {Query} this
 * @see MongoDB docs
https://docs.mongodb.com/manual/reference/method/cursor.collation/#cursor.collation
 * @api public
 */

Query.prototype.collation = function(value) {
  this.options.collation = value;
  return this;
};

/**
 * Specifies a `$where` condition
 *
 * Use `$where` when you need to select documents using a JavaScript expression.
 *
 * #### Example:
 *
 *     query.$where('this.comments.length > 10 || this.name.length > 5')
 *
 *     query.$where(function () {
 *       return this.comments.length > 10 || this.name.length > 5;
 *     })
 *
 * @param {String|Function} js javascript string or function
 * @return {Query} this
 * @memberOf Query
 * @method $where
 * @api public
 */

Query.prototype.$where = function(js) {
  this._conditions.$where = js;
  return this;
};

/**
 * Specifies a `path` for use with chaining.
 *
 * #### Example:
 *
 *     // instead of writing:
 *     await User.find({age: {$gte: 21, $lte: 65}});
 *
 *     // we can instead write:
 *     User.where('age').gte(21).lte(65);
 *
 *     // passing query conditions is permitted
 *     User.find().where({ name: 'vonderful' })
 *
 *     // chaining
 *     await User
 *       .where('age').gte(21).lte(65)
 *       .where('name', /^vonderful/i)
 *       .where('friends').slice(10)
 *       .exec()
 *
 * @param {String} [path]
 * @param {Object} [val]
 * @return {Query} this
```

```
 * @api public
 */

Query.prototype.where = function() {
  if (!arguments.length) return this;
  if (!this.op) this.op = 'find';

  const type = typeof arguments[0];

  if ('string' == type) {
    this._path = arguments[0];

    if (2 === arguments.length) {
      this._conditions[this._path] = arguments[1];
    }

    return this;
  }

  if ('object' == type && !Array.isArray(arguments[0])) {
    return this.merge(arguments[0]);
  }

  throw new TypeError('path must be a string or object');
};

/**
 * Specifies the complementary comparison value for paths specified with `where()`
 *
 * #### Example:
 *
 *     User.where('age').equals(49);
 *
 *     // is the same as
 *
 *     User.where('age', 49);
 *
 * @param {Object} val
 * @return {Query} this
 * @api public
 */

Query.prototype.equals = function equals(val) {
  this._ensurePath('equals');
  const path = this._path;
  this._conditions[path] = val;
  return this;
};

/**
 * Specifies the complementary comparison value for paths specified with `where()`
 * This is alias of `equals`
 *
 * #### Example:
 *
 *     User.where('age').eq(49);
 *
 *     // is the same as
 *
 *     User.shere('age').equals(49);
 *
 *     // is the same as
 *
 *     User.where('age', 49);
 *
 * @param {Object} val
 * @return {Query} this
 * @api public
 */
```

```
Query.prototype.eq = function eq(val) {
  this._ensurePath('eq');
  const path = this._path;
  this._conditions[path] = val;
  return this;
};

/**
 * Specifies arguments for an `$or` condition.
 *
 * #### Example:
 *
 *     query.or([{ color: 'red' }, { status: 'emergency' }])
 *
 * @param {Array} array array of conditions
 * @return {Query} this
 * @api public
 */

Query.prototype.or = function or(array) {
  const or = this._conditions.$or || (this._conditions.$or = []);
  if (!Array.isArray(array)) array = [array];
  or.push.apply(or, array);
  return this;
};

/**
 * Specifies arguments for a `$nor` condition.
 *
 * #### Example:
 *
 *     query.nor([{ color: 'green' }, { status: 'ok' }])
 *
 * @param {Array} array array of conditions
 * @return {Query} this
 * @api public
 */

Query.prototype.nor = function nor(array) {
  const nor = this._conditions.$nor || (this._conditions.$nor = []);
  if (!Array.isArray(array)) array = [array];
  nor.push.apply(nor, array);
  return this;
};

/**
 * Specifies arguments for a `$and` condition.
 *
 * #### Example:
 *
 *     query.and([{ color: 'green' }, { status: 'ok' }])
 *
 * @see $and http://docs.mongodb.org/manual/reference/operator/and/
 * @param {Array} array array of conditions
 * @return {Query} this
 * @api public
 */

Query.prototype.and = function and(array) {
  const and = this._conditions.$and || (this._conditions.$and = []);
  if (!Array.isArray(array)) array = [array];
  and.push.apply(and, array);
  return this;
};

/**
 * Specifies a $gt query condition.
 *
 * When called with one argument, the most recent path passed to `where()` is used.
 *
```

```
 * #### Example:
 *
 *     Thing.find().where('age').gt(21)
 *
 *     // or
 *     Thing.find().gt('age', 21)
 *
 * @method gt
 * @memberOf Query
 * @param {String} [path]
 * @param {Number} val
 * @api public
 */

/**
 * Specifies a $gte query condition.
 *
 * When called with one argument, the most recent path passed to `where()` is used.
 *
 * @method gte
 * @memberOf Query
 * @param {String} [path]
 * @param {Number} val
 * @api public
 */

/**
 * Specifies a $lt query condition.
 *
 * When called with one argument, the most recent path passed to `where()` is used.
 *
 * @method lt
 * @memberOf Query
 * @param {String} [path]
 * @param {Number} val
 * @api public
 */

/**
 * Specifies a $lte query condition.
 *
 * When called with one argument, the most recent path passed to `where()` is used.
 *
 * @method lte
 * @memberOf Query
 * @param {String} [path]
 * @param {Number} val
 * @api public
 */

/**
 * Specifies a $ne query condition.
 *
 * When called with one argument, the most recent path passed to `where()` is used.
 *
 * @method ne
 * @memberOf Query
 * @param {String} [path]
 * @param {Number} val
 * @api public
 */

/**
 * Specifies an $in query condition.
 *
 * When called with one argument, the most recent path passed to `where()` is used.
 *
 * @method in
 * @memberOf Query
 * @param {String} [path]
```

```
 * @param {Number} val
 * @api public
 */

/**
 * Specifies an $nin query condition.
 *
 * When called with one argument, the most recent path passed to `where()` is used.
 *
 * @method nin
 * @memberOf Query
 * @param {String} [path]
 * @param {Number} val
 * @api public
 */

/**
 * Specifies an $all query condition.
 *
 * When called with one argument, the most recent path passed to `where()` is used.
 *
 * @method all
 * @memberOf Query
 * @param {String} [path]
 * @param {Number} val
 * @api public
 */

/**
 * Specifies a $size query condition.
 *
 * When called with one argument, the most recent path passed to `where()` is used.
 *
 * @method size
 * @memberOf Query
 * @param {String} [path]
 * @param {Number} val
 * @api public
 */

/**
 * Specifies a $regex query condition.
 *
 * When called with one argument, the most recent path passed to `where()` is used.
 *
 * @method regex
 * @memberOf Query
 * @param {String} [path]
 * @param {String|RegExp} val
 * @api public
 */

/**
 * Specifies a $maxDistance query condition.
 *
 * When called with one argument, the most recent path passed to `where()` is used.
 *
 * @method maxDistance
 * @memberOf Query
 * @param {String} [path]
 * @param {Number} val
 * @api public
 */

/*!
 * gt, gte, lt, lte, ne, in, nin, all, regex, size, maxDistance
 *
 *     Thing.where('type').nin(array)
 */
```

```
'gt gte lt lte ne in nin all regex size maxDistance minDistance'.split('
').forEach(function($conditional) {
  Query.prototype[$conditional] = function() {
    let path, val;

    if (1 === arguments.length) {
      this._ensurePath($conditional);
      val = arguments[0];
      path = this._path;
    } else {
      val = arguments[1];
      path = arguments[0];
    }

    const conds = this._conditions[path] === null || typeof this._conditions[path] === 'object' ?
      this._conditions[path] :
      (this._conditions[path] = {});
    conds['$' + $conditional] = val;
    return this;
  };
});

/**
 * Specifies a `$mod` condition
 *
 * @param {String} [path]
 * @param {Number} val
 * @return {Query} this
 * @api public
 */

Query.prototype.mod = function() {
  let val, path;

  if (1 === arguments.length) {
    this._ensurePath('mod');
    val = arguments[0];
    path = this._path;
  } else if (2 === arguments.length && !Array.isArray(arguments[1])) {
    this._ensurePath('mod');
    val = [arguments[0], arguments[1]];
    path = this._path;
  } else if (3 === arguments.length) {
    val = [arguments[1], arguments[2]];
    path = arguments[0];
  } else {
    val = arguments[1];
    path = arguments[0];
  }

  const conds = this._conditions[path] || (this._conditions[path] = {});
  conds.$mod = val;
  return this;
};

/**
 * Specifies an `$exists` condition
 *
 * #### Example:
 *
 *     // { name: { $exists: true }}
 *     Thing.where('name').exists()
 *     Thing.where('name').exists(true)
 *     Thing.find().exists('name')
 *
 *     // { name: { $exists: false }}
 *     Thing.where('name').exists(false);
 *     Thing.find().exists('name', false);
 *
 * @param {String} [path]
```

```
 * @param {Number} val
 * @return {Query} this
 * @api public
 */

Query.prototype.exists = function() {
  let path, val;

  if (0 === arguments.length) {
    this._ensurePath('exists');
    path = this._path;
    val = true;
  } else if (1 === arguments.length) {
    if ('boolean' === typeof arguments[0]) {
      this._ensurePath('exists');
      path = this._path;
      val = arguments[0];
    } else {
      path = arguments[0];
      val = true;
    }
  } else if (2 === arguments.length) {
    path = arguments[0];
    val = arguments[1];
  }

  const conds = this._conditions[path] || (this._conditions[path] = {});
  conds.$exists = val;
  return this;
};

/**
 * Specifies an `$elemMatch` condition
 *
 * #### Example:
 *
 *     query.elemMatch('comment', { author: 'autobot', votes: {$gte: 5}})
 *
 *     query.where('comment').elemMatch({ author: 'autobot', votes: {$gte: 5}})
 *
 *     query.elemMatch('comment', function (elem) {
 *       elem.where('author').equals('autobot');
 *       elem.where('votes').gte(5);
 *     })
 *
 *     query.where('comment').elemMatch(function  (elem)  {
 *       elem.where({ author: 'autobot' });
 *       elem.where('votes').gte(5);
 *     })
 *
 * @param {String|Object|Function} path
 * @param {Object|Function} criteria
 * @return {Query} this
 * @api public
 */

Query.prototype.elemMatch = function() {
  if (null == arguments[0])
    throw new TypeError('Invalid argument');

  let fn, path, criteria;

  if ('function' === typeof arguments[0]) {
    this._ensurePath('elemMatch');
    path = this._path;
    fn = arguments[0];
  } else if (utils.isObject(arguments[0])) {
    this._ensurePath('elemMatch');
    path = this._path;
    criteria = arguments[0];
```

```
  } else if ('function' === typeof arguments[1]) {
    path = arguments[0];
    fn = arguments[1];
  } else if (arguments[1] && utils.isObject(arguments[1])) {
    path = arguments[0];
    criteria = arguments[1];
  } else {
    throw new TypeError('Invalid argument');
  }

  if (fn) {
    criteria = new Query;
    fn(criteria);
    criteria = criteria._conditions;
  }

  const conds = this._conditions[path] || (this._conditions[path] = {});
  conds.$elemMatch = criteria;
  return this;
};

// Spatial queries

/**
 * Sugar for geo-spatial queries.
 *
 * #### Example:
 *
 *     query.within().box()
 *     query.within().circle()
 *     query.within().geometry()
 *
 *     query.where('loc').within({ center: [50,50], radius: 10, unique: true, spherical: true });
 *     query.where('loc').within({ box: [[40.73, -73.9], [40.7, -73.988]] });
 *     query.where('loc').within({ polygon: [[],[],[],[]] });
 *
 *     query.where('loc').within([], [], []) // polygon
 *     query.where('loc').within([], []) // box
 *     query.where('loc').within({ type: 'LineString', coordinates: [...] }); // geometry
 *
 * #### Note:
 *
 * Must be used after `where()`.
 *
 * @memberOf Query
 * @return {Query} this
 * @api public
 */

Query.prototype.within = function within() {
  // opinionated, must be used after where
  this._ensurePath('within');
  this._geoComparison = '$geoWithin';

  if (0 === arguments.length) {
    return this;
  }

  if (2 === arguments.length) {
    return this.box.apply(this, arguments);
  } else if (2 < arguments.length) {
    return this.polygon.apply(this, arguments);
  }

  const area = arguments[0];

  if (!area)
    throw new TypeError('Invalid argument');

  if (area.center)
```

```
    return this.circle(area);

  if (area.box)
    return this.box.apply(this, area.box);

  if (area.polygon)
    return this.polygon.apply(this, area.polygon);

  if (area.type && area.coordinates)
    return this.geometry(area);

  throw new TypeError('Invalid argument');
};

/**
 * Specifies a $box condition
 *
 * #### Example:
 *
 *     var lowerLeft = [40.73083, -73.99756]
 *     var upperRight= [40.741404,  -73.988135]
 *
 *     query.where('loc').within().box(lowerLeft,  upperRight)
 *     query.box('loc', lowerLeft, upperRight )
 *
 * @see http://www.mongodb.org/display/DOCS/Geospatial+Indexing
 * @see Query#within #query_Query-within
 * @param {String} path
 * @param {Object} val
 * @return {Query} this
 * @api public
 */

Query.prototype.box = function() {
  let path, box;

  if (3 === arguments.length) {
    // box('loc', [], [])
    path = arguments[0];
    box = [arguments[1], arguments[2]];
  } else if (2 === arguments.length) {
    // box([], [])
    this._ensurePath('box');
    path = this._path;
    box = [arguments[0], arguments[1]];
  } else {
    throw new TypeError('Invalid argument');
  }

  const conds = this._conditions[path] || (this._conditions[path] = {});
  conds[this._geoComparison || '$geoWithin'] = { $box: box };
  return this;
};

/**
 * Specifies a $polygon condition
 *
 * #### Example:
 *
 *     query.where('loc').within().polygon([10,20], [13, 25], [7,15])
 *     query.polygon('loc', [10,20], [13, 25], [7,15])
 *
 * @param {String|Array} [path]
 * @param {Array|Object} [val]
 * @return {Query} this
 * @see http://www.mongodb.org/display/DOCS/Geospatial+Indexing
 * @api public
 */

Query.prototype.polygon = function() {
```

```
  let val, path;

  if ('string' == typeof arguments[0]) {
    // polygon('loc', [],[],[])
    val = Array.from(arguments);
    path = val.shift();
  } else {
    // polygon([],[],[])
    this._ensurePath('polygon');
    path = this._path;
    val = Array.from(arguments);
  }

  const conds = this._conditions[path] || (this._conditions[path] = {});
  conds[this._geoComparison || '$geoWithin'] = { $polygon: val };
  return this;
};

/**
 * Specifies a $center or $centerSphere condition.
 *
 * #### Example:
 *
 *     var area = { center: [50, 50], radius: 10, unique: true }
 *     query.where('loc').within().circle(area)
 *     query.center('loc', area);
 *
 *     // for spherical calculations
 *     var area = { center: [50, 50], radius: 10, unique: true, spherical: true }
 *     query.where('loc').within().circle(area)
 *     query.center('loc', area);
 *
 * @param {String} [path]
 * @param {Object} area
 * @return {Query} this
 * @see http://www.mongodb.org/display/DOCS/Geospatial+Indexing
 * @api public
 */

Query.prototype.circle = function() {
  let path, val;

  if (1 === arguments.length) {
    this._ensurePath('circle');
    path = this._path;
    val = arguments[0];
  } else if (2 === arguments.length) {
    path = arguments[0];
    val = arguments[1];
  } else {
    throw new TypeError('Invalid argument');
  }

  if (!('radius' in val && val.center))
    throw new Error('center and radius are required');

  const conds = this._conditions[path] || (this._conditions[path] = {});

  const type = val.spherical
    ? '$centerSphere'
    : '$center';

  const wKey = this._geoComparison || '$geoWithin';
  conds[wKey] = {};
  conds[wKey][type] = [val.center, val.radius];

  if ('unique' in val)
    conds[wKey].$uniqueDocs = !!val.unique;

  return this;
```

```
};

/**
 * Specifies a `$near` or `$nearSphere` condition
 *
 * These operators return documents sorted by distance.
 *
 * #### Example:
 *
 *     query.where('loc').near({ center: [10, 10] });
 *     query.where('loc').near({ center: [10, 10], maxDistance: 5 });
 *     query.where('loc').near({ center: [10, 10], maxDistance: 5, spherical: true });
 *     query.near('loc', { center: [10, 10], maxDistance: 5 });
 *     query.near({ center: { type: 'Point', coordinates: [..] }})
 *     query.near().geometry({ type: 'Point', coordinates: [..] })
 *
 * @param {String} [path]
 * @param {Object} val
 * @return {Query} this
 * @see http://www.mongodb.org/display/DOCS/Geospatial+Indexing
 * @api public
 */

Query.prototype.near = function near() {
  let path, val;

  this._geoComparison = '$near';

  if (0 === arguments.length) {
    return this;
  } else if (1 === arguments.length) {
    this._ensurePath('near');
    path = this._path;
    val = arguments[0];
  } else if (2 === arguments.length) {
    path = arguments[0];
    val = arguments[1];
  } else {
    throw new TypeError('Invalid argument');
  }

  if (!val.center) {
    throw new Error('center is required');
  }

  const conds = this._conditions[path] || (this._conditions[path] = {});

  const type = val.spherical
    ? '$nearSphere'
    : '$near';

  // center could be a GeoJSON object or an Array
  if (Array.isArray(val.center)) {
    conds[type] = val.center;

    const radius = 'maxDistance' in val
      ? val.maxDistance
      : null;

    if (null != radius) {
      conds.$maxDistance = radius;
    }
    if (null != val.minDistance) {
      conds.$minDistance = val.minDistance;
    }
  } else {
    // GeoJSON?
    if (val.center.type != 'Point' || !Array.isArray(val.center.coordinates)) {
      throw new Error(util.format('Invalid GeoJSON specified for %s', type));
    }
```

```
    conds[type] = { $geometry: val.center };

    // MongoDB 2.6 insists on maxDistance being in $near / $nearSphere
    if ('maxDistance' in val) {
      conds[type]['$maxDistance'] = val.maxDistance;
    }
    if ('minDistance' in val) {
      conds[type]['$minDistance'] = val.minDistance;
    }
  }

  return this;
};

/**
 * Declares an intersects query for `geometry()`.
 *
 * #### Example:
 *
 *     query.where('path').intersects().geometry({
 *         type: 'LineString'
 *       , coordinates: [[180.0, 11.0], [180, 9.0]]
 *     })
 *
 *     query.where('path').intersects({
 *         type: 'LineString'
 *       , coordinates: [[180.0, 11.0], [180, 9.0]]
 *     })
 *
 * @param {Object} [arg]
 * @return {Query} this
 * @api public
 */

Query.prototype.intersects = function intersects() {
  // opinionated, must be used after where
  this._ensurePath('intersects');

  this._geoComparison = '$geoIntersects';

  if (0 === arguments.length) {
    return this;
  }

  const area = arguments[0];

  if (null != area && area.type && area.coordinates)
    return this.geometry(area);

  throw new TypeError('Invalid argument');
};

/**
 * Specifies a `$geometry` condition
 *
 * #### Example:
 *
 *     var polyA = [[[ 10, 20 ], [ 10, 40 ], [ 30, 40 ], [ 30, 20 ]]]
 *     query.where('loc').within().geometry({ type: 'Polygon', coordinates: polyA })
 *
 *     // or
 *     var polyB = [[ 0, 0 ], [ 1, 1 ]]
 *     query.where('loc').within().geometry({ type: 'LineString', coordinates: polyB })
 *
 *     // or
 *     var polyC = [ 0, 0 ]
 *     query.where('loc').within().geometry({ type: 'Point', coordinates: polyC })
 *
 *     // or
 *     query.where('loc').intersects().geometry({ type: 'Point', coordinates: polyC })
```

```
 *
 * #### Note:
 *
 * `geometry()` **must** come after either `intersects()` or `within()`.
 *
 * The `object` argument must contain `type` and `coordinates` properties.
 * - type {String}
 * - coordinates {Array}
 *
 * The most recent path passed to `where()` is used.
 *
 * @param {Object} object Must contain a `type` property which is a String and a
`coordinates` property which is an Array. See the examples.
 * @return {Query} this
 * @see http://docs.mongodb.org/manual/release-notes/2.4/#new-geospatial-indexes-with-
geojson-and-improved-spherical-geometry
 * @see http://www.mongodb.org/display/DOCS/Geospatial+Indexing
 * @see $geometry http://docs.mongodb.org/manual/reference/operator/geometry/
 * @api public
 */

Query.prototype.geometry = function geometry() {
  if (!('$within' == this._geoComparison ||
        '$geoWithin' == this._geoComparison ||
        '$near' == this._geoComparison ||
        '$geoIntersects' == this._geoComparison)) {
    throw new Error('geometry() must come after `within()`, `intersects()`, or `near()');
  }

  let val, path;

  if (1 === arguments.length) {
    this._ensurePath('geometry');
    path = this._path;
    val = arguments[0];
  } else {
    throw new TypeError('Invalid argument');
  }

  if (!(val.type && Array.isArray(val.coordinates))) {
    throw new TypeError('Invalid argument');
  }

  const conds = this._conditions[path] || (this._conditions[path] = {});
  conds[this._geoComparison] = { $geometry: val };

  return this;
};

// end spatial

/**
 * Specifies which document fields to include or exclude
 *
 * #### String syntax
 *
 * When passing a string, prefixing a path with `-` will flag that path as excluded. When a path
does not have the `-` prefix, it is included.
 *
 * #### Example:
 *
 *     // include a and b, exclude c
 *     query.select('a b -c');
 *
 *     // or you may use object notation, useful when
 *     // you have keys already prefixed with a "-"
 *     query.select({a: 1, b: 1, c: 0});
 *
 * #### Note:
 *
```

```
 * Cannot be used with `distinct()`
 *
 * @param {Object|String} arg
 * @return {Query} this
 * @see SchemaType
 * @api public
 */

Query.prototype.select = function select() {
  let arg = arguments[0];
  if (!arg) return this;

  if (arguments.length !== 1) {
    throw new Error('Invalid select: select only takes 1 argument');
  }

  this._validate('select');

  const fields = this._fields || (this._fields = {});
  const type = typeof arg;
  let i, len;

  if (('string' == type || utils.isArgumentsObject(arg)) &&
    'number' == typeof arg.length || Array.isArray(arg)) {
    if ('string' == type)
      arg = arg.split(/\s+/);

    for (i = 0, len = arg.length; i < len; ++i) {
      let field = arg[i];
      if (!field) continue;
      const include = '-' == field[0] ? 0 : 1;
      if (include === 0) field = field.substring(1);
      fields[field] = include;
    }

    return this;
  }

  if (utils.isObject(arg)) {
    const keys = utils.keys(arg);
    for (i = 0; i < keys.length; ++i) {
      fields[keys[i]] = arg[keys[i]];
    }
    return this;
  }

  throw new TypeError('Invalid select() argument. Must be string or object.');
};

/**
 * Specifies a $slice condition for a `path`
 *
 * #### Example:
 *
 *     query.slice('comments', 5)
 *     query.slice('comments', -5)
 *     query.slice('comments', [10, 5])
 *     query.where('comments').slice(5)
 *     query.where('comments').slice([-10, 5])
 *
 * @param {String} [path]
 * @param {Number} val number/range of elements to slice
 * @return {Query} this
 * @see mongodb
http://www.mongodb.org/display/DOCS/Retrieving+a+Subset+of+Fields#RetrievingaSubsetofFields-RetrievingaSubrangeofArrayElements
 * @api public
 */

Query.prototype.slice = function() {
```

```
  if (0 === arguments.length)
    return this;

  this._validate('slice');

  let path, val;

  if (1 === arguments.length) {
    const arg = arguments[0];
    if (typeof arg === 'object' && !Array.isArray(arg)) {
      const keys = Object.keys(arg);
      const numKeys = keys.length;
      for (let i = 0; i < numKeys; ++i) {
      this.slice(keys[i], arg[keys[i]]);
      }
      return this;
    }
    this._ensurePath('slice');
    path = this._path;
    val = arguments[0];
  } else if (2 === arguments.length) {
    if ('number' === typeof arguments[0]) {
      this._ensurePath('slice');
      path = this._path;
      val = [arguments[0], arguments[1]];
    } else {
      path = arguments[0];
      val = arguments[1];
    }
  } else if (3 === arguments.length) {
    path = arguments[0];
    val = [arguments[1], arguments[2]];
  }

  const myFields = this._fields || (this._fields = {});
  myFields[path] = { $slice: val };
  return this;
};

/**
 * Sets the sort order
 *
 * If an object is passed, values allowed are 'asc', 'desc', 'ascending', 'descending', 1, and -1.
 *
 * If a string is passed, it must be a space delimited list of path names. The sort order of each
path is ascending unless the path name is prefixed with `-` which will be treated as
descending.
 *
 * #### Example:
 *
 *     // these are equivalent
 *     query.sort({ field: 'asc', test: -1 });
 *     query.sort('field -test');
 *     query.sort([['field', 1], ['test', -1]]);
 *
 * #### Note:
 *
 *  - The array syntax `.sort([['field', 1], ['test', -1]])` can only be used with [mongodb driver
>= 2.0.46](https://github.com/mongodb/node-mongodb-native/blob/2.1/HISTORY.md#2046-
2015-10-15).
 *  - Cannot be used with `distinct()`
 *
 * @param {Object|String|Array} arg
 * @return {Query} this
 * @api public
 */

Query.prototype.sort = function(arg) {
  if (!arg) return this;
  let i, len, field;
```

```
  this._validate('sort');

  const type = typeof arg;

  // .sort([['field', 1], ['test', -1]])
  if (Array.isArray(arg)) {
    len = arg.length;
    for (i = 0; i < arg.length; ++i) {
      if (!Array.isArray(arg[i])) {
        throw new Error('Invalid sort() argument, must be array of arrays');
      }
      _pushArr(this.options, arg[i][0], arg[i][1]);
    }
    return this;
  }

  // .sort('field -test')
  if (1 === arguments.length && 'string' == type) {
    arg = arg.split(/\s+/);
    len = arg.length;
    for (i = 0; i < len; ++i) {
      field = arg[i];
      if (!field) continue;
      const ascend = '-' == field[0] ? -1 : 1;
      if (ascend === -1) field = field.substring(1);
      push(this.options, field, ascend);
    }

    return this;
  }

  // .sort({ field: 1, test: -1 })
  if (utils.isObject(arg)) {
    const keys = utils.keys(arg);
    for (i = 0; i < keys.length; ++i) {
      field = keys[i];
      push(this.options, field, arg[field]);
    }

    return this;
  }

  if (typeof Map !== 'undefined' && arg instanceof Map) {
    _pushMap(this.options, arg);
    return this;
  }
  throw new TypeError('Invalid sort() argument. Must be a string, object, or array.');
};

/*!
 * @ignore
 */

const _validSortValue = {
  1: 1,
  '-1': -1,
  asc: 1,
  ascending: 1,
  desc: -1,
  descending: -1
};

function push(opts, field, value) {
  if (Array.isArray(opts.sort)) {
    throw new TypeError('Can\'t mix sort syntaxes. Use either array or object:' +
      '\n- `.sort([[\'field\', 1], [\'test\', -1]])`' +
      '\n- `.sort({ field: 1, test: -1 })`');
  }
```

```
  let s;
  if (value && value.$meta) {
    s = opts.sort || (opts.sort = {});
    s[field] = { $meta: value.$meta };
    return;
  }

  s = opts.sort || (opts.sort = {});
  let val = String(value || 1).toLowerCase();
  val = _validSortValue[val];
  if (!val) throw new TypeError('Invalid sort value: { ' + field + ': ' + value + ' }');

  s[field] = val;
}

function _pushArr(opts, field, value) {
  opts.sort = opts.sort || [];
  if (!Array.isArray(opts.sort)) {
    throw new TypeError('Can\'t mix sort syntaxes. Use either array or object:' +
      '\n- `.sort([[\'field\', 1], [\'test\', -1]])`' +
      '\n- `.sort({ field: 1, test: -1 })`');
  }

  let val = String(value || 1).toLowerCase();
  val = _validSortValue[val];
  if (!val) throw new TypeError('Invalid sort value: [ ' + field + ', ' + value + ' ]');

  opts.sort.push([field, val]);
}

function _pushMap(opts, map) {
  opts.sort = opts.sort || new Map();
  if (!(opts.sort instanceof Map)) {
    throw new TypeError('Can\'t mix sort syntaxes. Use either array or ' +
      'object or map consistently');
  }
  map.forEach(function(value, key) {
    let val = String(value || 1).toLowerCase();
    val = _validSortValue[val];
    if (!val) throw new TypeError('Invalid sort value: < ' + key + ': ' + value + ' >');

    opts.sort.set(key, val);
  });
}


/**
 * Specifies the limit option.
 *
 * #### Example:
 *
 *     query.limit(20)
 *
 * #### Note:
 *
 * Cannot be used with `distinct()`
 *
 * @method limit
 * @memberOf Query
 * @param {Number} val
 * @see mongodb
http://www.mongodb.org/display/DOCS/Advanced+Queries#AdvancedQueries-
%7B%7Blimit%28%29%7D%7D
 * @api public
 */
/**
 * Specifies the skip option.
 *
 * #### Example:
 *
```

```
 *     query.skip(100).limit(20)
 *
 * #### Note:
 *
 * Cannot be used with `distinct()`
 *
 * @method skip
 * @memberOf Query
 * @param {Number} val
 * @see mongodb
http://www.mongodb.org/display/DOCS/Advanced+Queries#AdvancedQueries-
%7B%7Bskip%28%29%7D%7D
 * @api public
 */
/**
 * Specifies the batchSize option.
 *
 * #### Example:
 *
 *     query.batchSize(100)
 *
 * #### Note:
 *
 * Cannot be used with `distinct()`
 *
 * @method batchSize
 * @memberOf Query
 * @param {Number} val
 * @see mongodb
http://www.mongodb.org/display/DOCS/Advanced+Queries#AdvancedQueries-
%7B%7BbatchSize%28%29%7D%7D
 * @api public
 */
/**
 * Specifies the `comment` option.
 *
 * #### Example:
 *
 *     query.comment('login query')
 *
 * #### Note:
 *
 * Cannot be used with `distinct()`
 *
 * @method comment
 * @memberOf Query
 * @param {Number} val
 * @see mongodb
http://www.mongodb.org/display/DOCS/Advanced+Queries#AdvancedQueries-
%24comment
 * @api public
 */

/*!
 * limit, skip, batchSize, comment
 *
 * Sets these associated options.
 *
 *     query.comment('feed query');
 */

['limit', 'skip', 'batchSize', 'comment'].forEach(function(method) {
  Query.prototype[method] = function(v) {
  this._validate(method);
    this.options[method] = v;
    return this;
  };
});

/**
```

```
 * Specifies the maxTimeMS option.
 *
 * #### Example:
 *
 *     query.maxTime(100)
 *     query.maxTimeMS(100)
 *
 * @method maxTime
 * @memberOf Query
 * @param {Number} ms
 * @see mongodb
http://docs.mongodb.org/manual/reference/operator/meta/maxTimeMS/#op._S_maxTimeMS
 * @api public
 */

Query.prototype.maxTime = Query.prototype.maxTimeMS = function(ms) {
  this._validate('maxTime');
  this.options.maxTimeMS = ms;
  return this;
};

/**
 * Sets query hints.
 *
 * #### Example:
 *
 *     query.hint({ indexA: 1, indexB: -1});
 *     query.hint('indexA_1_indexB_1');
 *
 * #### Note:
 *
 * Cannot be used with `distinct()`
 *
 * @param {Object|string} val a hint object or the index name
 * @return {Query} this
 * @see mongodb
http://www.mongodb.org/display/DOCS/Advanced+Queries#AdvancedQueries-%24hint
 * @api public
 */

Query.prototype.hint = function() {
  if (0 === arguments.length) return this;

  this._validate('hint');

  const arg = arguments[0];
  if (utils.isObject(arg)) {
    const hint = this.options.hint || (this.options.hint = {});

    // must keep object keys in order so don't use Object.keys()
    for (const k in arg) {
      hint[k] = arg[k];
    }

    return this;
  }
  if (typeof arg === 'string') {
    this.options.hint = arg;
    return this;
  }

  throw new TypeError('Invalid hint. ' + arg);
};

/**
 * Requests acknowledgement that this operation has been persisted to MongoDB's
 * on-disk journal.
 * This option is only valid for operations that write to the database:
 *
 * - `deleteOne()`
```

```
 * - `deleteMany()`
 * - `findOneAndDelete()`
 * - `findOneAndUpdate()`
 * - `updateOne()`
 * - `updateMany()`
 *
 * Defaults to the `j` value if it is specified in writeConcern options
 *
 * #### Example:
 *
 *     mquery().w(2).j(true).wtimeout(2000);
 *
 * @method j
 * @memberOf Query
 * @instance
 * @param {boolean} val
 * @see mongodb https://docs.mongodb.com/manual/reference/write-concern/#j-option
 * @return {Query} this
 * @api public
 */

Query.prototype.j = function j(val) {
  this.options.j = val;
  return this;
};

/**
 * Sets the slaveOk option. _Deprecated_ in MongoDB 2.2 in favor of read preferences.
 *
 * #### Example:
 *
 *     query.slaveOk() // true
 *     query.slaveOk(true)
 *     query.slaveOk(false)
 *
 * @deprecated use read() preferences instead if on mongodb >= 2.2
 * @param {Boolean} v defaults to true
 * @see  mongodb  http://docs.mongodb.org/manual/applications/replication/#read-preference
 * @see read()
 * @return {Query} this
 * @api public
 */

Query.prototype.slaveOk = function(v) {
  this.options.slaveOk = arguments.length ? !!v : true;
  return this;
};

/**
 * Sets the readPreference option for the query.
 *
 * #### Example:
 *
 *     new Query().read('primary')
 *     new Query().read('p')  // same as primary
 *
 *     new Query().read('primaryPreferred')
 *     new Query().read('pp') // same as primaryPreferred
 *
 *     new Query().read('secondary')
 *     new Query().read('s')  // same as secondary
 *
 *     new Query().read('secondaryPreferred')
 *     new Query().read('sp') // same as secondaryPreferred
 *
 *     new Query().read('nearest')
 *     new Query().read('n')  // same as nearest
 *
 *     // you can also use mongodb.ReadPreference class to also specify tags
 *     new Query().read(mongodb.ReadPreference('secondary', [{ dc:'sf', s: 1 },{ dc:'ma', s: 2
```

```
}]))
 *
 *     new Query().setReadPreference('primary') // alias of .read()
 *
 * #### Preferences:
 *
 *     primary - (default)  Read from primary only. Operations will produce an error if primary
is unavailable. Cannot be combined with tags.
 *     secondary            Read from secondary if available, otherwise error.
 *     primaryPreferred     Read from primary if available, otherwise a secondary.
 *     secondaryPreferred   Read from a secondary if available, otherwise read from the
primary.
 *     nearest              All operations read from among the nearest candidates, but unlike other
modes, this option will include both the primary and all secondaries in the random selection.
 *
 * Aliases
 *
 *     p   primary
 *     pp  primaryPreferred
 *     s   secondary
 *     sp  secondaryPreferred
 *     n   nearest
 *
 * Read more about how to use read preferences
[here](http://docs.mongodb.org/manual/applications/replication/#read-preference) and
[here](http://mongodb.github.com/node-mongodb-native/driver-
articles/anintroductionto1_1and2_2.html#read-preferences).
 *
 * @param {String|ReadPreference} pref one of the listed preference options or their aliases
 * @see  mongodb  http://docs.mongodb.org/manual/applications/replication/#read-preference
 * @see driver http://mongodb.github.com/node-mongodb-native/driver-
articles/anintroductionto1_1and2_2.html#read-preferences
 * @return {Query} this
 * @api public
 */

Query.prototype.read = Query.prototype.setReadPreference = function(pref) {
  if (arguments.length > 1 && !Query.prototype.read.deprecationWarningIssued) {
    console.error('Deprecation warning: \'tags\' argument is not supported anymore in
Query.read() method. Please use mongodb.ReadPreference object instead.');
    Query.prototype.read.deprecationWarningIssued = true;
  }
  this.options.readPreference = utils.readPref(pref);
  return this;
};

/**
 * Sets the readConcern option for the query.
 *
 * #### Example:
 *
 *     new Query().readConcern('local')
 *     new Query().readConcern('l')  // same as local
 *
 *     new Query().readConcern('available')
 *     new Query().readConcern('a')  // same as available
 *
 *     new Query().readConcern('majority')
 *     new Query().readConcern('m')  // same as majority
 *
 *     new Query().readConcern('linearizable')
 *     new Query().readConcern('lz') // same as linearizable
 *
 *     new Query().readConcern('snapshot')
 *     new Query().readConcern('s')  // same as snapshot
 *
 *     new Query().r('s') // r is alias of readConcern
 *
 *
 * #### Read Concern Level:
```

```
 *
 *     local         MongoDB 3.2+ The query returns from the instance with no guarantee
guarantee that the data has been written to a majority of the replica set members (i.e. may be
rolled back).
 *     available     MongoDB 3.6+ The query returns from the instance with no guarantee
guarantee that the data has been written to a majority of the replica set members (i.e. may be
rolled back).
 *     majority      MongoDB 3.2+ The query returns the data that has been acknowledged by a
majority of the replica set members. The documents returned by the read operation are
durable, even in the event of failure.
 *     linearizable  MongoDB 3.4+ The query returns data that reflects all successful majority-
acknowledged writes that completed prior to the start of the read operation. The query may
wait for concurrently executing writes to propagate to a majority of replica set members
before returning results.
 *     snapshot      MongoDB 4.0+ Only available for operations within multi-document
transactions. Upon transaction commit with write concern "majority", the transaction
operations are guaranteed to have read from a snapshot of majority-committed data.
 *
 * Aliases
 *
 *     l   local
 *     a   available
 *     m   majority
 *     lz  linearizable
 *     s   snapshot
 *
 * Read more about how to use read concern
[here](https://docs.mongodb.com/manual/reference/read-concern/).
 *
 * @param {String} level one of the listed read concern level or their aliases
 * @see mongodb https://docs.mongodb.com/manual/reference/read-concern/
 * @return {Query} this
 * @api public
 */

Query.prototype.readConcern = Query.prototype.r = function(level) {
  this.options.readConcern = utils.readConcern(level);
  return this;
};

/**
 * Sets tailable option.
 *
 * #### Example:
 *
 *     query.tailable() <== true
 *     query.tailable(true)
 *     query.tailable(false)
 *
 * #### Note:
 *
 * Cannot be used with `distinct()`
 *
 * @param {Boolean} v defaults to true
 * @see mongodb http://www.mongodb.org/display/DOCS/Tailable+Cursors
 * @api public
 */

Query.prototype.tailable = function() {
  this._validate('tailable');

  this.options.tailable = arguments.length
    ? !!arguments[0]
    : true;

  return this;
};

/**
 * Sets the specified number of `mongod` servers, or tag set of `mongod` servers,
```

```
 * that must acknowledge this write before this write is considered successful.
 * This option is only valid for operations that write to the database:
 *
 * - `deleteOne()`
 * - `deleteMany()`
 * - `findOneAndDelete()`
 * - `findOneAndUpdate()`
 * - `updateOne()`
 * - `updateMany()`
 *
 * Defaults to the `w` value if it is specified in writeConcern options
 *
 * #### Example:
 *
 *     mquery().writeConcern(0)
 *     mquery().writeConcern(1)
 *     mquery().writeConcern({ w: 1, j: true, wtimeout: 2000 })
 *     mquery().writeConcern('majority')
 *     mquery().writeConcern('m') // same as majority
 *     mquery().writeConcern('tagSetName') // if the tag set is 'm', use .writeConcern({ w: 'm'
}) instead
 *     mquery().w(1) // w is alias of writeConcern
 *
 * @method writeConcern
 * @memberOf Query
 * @instance
 * @param {String|number|object} concern 0 for fire-and-forget, 1 for acknowledged by one
server, 'majority' for majority of the replica set, or [any of the more advanced
options](https://docs.mongodb.com/manual/reference/write-concern/#w-option).
 * @see  mongodb  https://docs.mongodb.com/manual/reference/write-concern/#w-option
 * @return {Query} this
 * @api public
 */

Query.prototype.writeConcern = Query.prototype.w = function writeConcern(concern) {
  if ('object' === typeof concern) {
    if ('undefined' !== typeof concern.j) this.options.j = concern.j;
    if ('undefined' !== typeof concern.w) this.options.w = concern.w;
    if ('undefined' !== typeof concern.wtimeout) this.options.wtimeout = concern.wtimeout;
  } else {
    this.options.w = 'm' === concern ? 'majority' : concern;
  }
  return this;
};

/**
 * Specifies a time limit, in milliseconds, for the write concern.
 * If `ms > 1`, it is maximum amount of time to wait for this write
 * to propagate through the replica set before this operation fails.
 * The default is `0`, which means no timeout.
 *
 * This option is only valid for operations that write to the database:
 *
 * - `deleteOne()`
 * - `deleteMany()`
 * - `findOneAndDelete()`
 * - `findOneAndUpdate()`
 * - `updateOne()`
 * - `updateMany()`
 *
 * Defaults to `wtimeout` value if it is specified in writeConcern
 *
 * #### Example:
 *
 *     mquery().w(2).j(true).wtimeout(2000)
 *
 * @method wtimeout
 * @memberOf Query
 * @instance
 * @param {number} ms number of milliseconds to wait
```

```
 * @see mongodb https://docs.mongodb.com/manual/reference/write-concern/#wtimeout
 * @return {Query} this
 * @api public
 */

Query.prototype.wtimeout = Query.prototype.wTimeout = function wtimeout(ms) {
  this.options.wtimeout = ms;
  return this;
};

/**
 * Merges another Query or conditions object into this one.
 *
 * When a Query is passed, conditions, field selection and options are merged.
 *
 * @param {Query|Object} source
 * @return {Query} this
 */

Query.prototype.merge = function(source) {
  if (!source)
    return this;

  if (!Query.canMerge(source))
    throw new TypeError('Invalid argument. Expected instanceof mquery or plain object');

  if (source instanceof Query) {
    // if source has a feature, apply it to ourselves

    if (source._conditions) {
      utils.merge(this._conditions, source._conditions);
    }

    if (source._fields) {
      this._fields || (this._fields = {});
      utils.merge(this._fields, source._fields);
    }

    if (source.options) {
      this.options || (this.options = {});
      utils.merge(this.options, source.options);
    }

    if (source._updateDoc) {
      this._updateDoc || (this._updateDoc = {});
      utils.mergeClone(this._updateDoc, source._updateDoc);
    }

    if (source._distinctDoc) {
      this._distinctDoc = source._distinctDoc;
    }

    return this;
  }

  // plain object
  utils.merge(this._conditions, source);

  return this;
};

/**
 * Finds documents.
 *
 * #### Example:
 *
 *     query.find()
 *     await query.find()
 *     await query.find({ name: 'Burning Lights' })
 *
```

```
 * @param {Object} [criteria] mongodb selector
 * @return {Query} this
 * @api public
 */

Query.prototype.find = function(criteria) {
  this.op = 'find';

  if (Query.canMerge(criteria)) {
    this.merge(criteria);
  }

  return this;
};

/**
 * Executes a `find` Query
 * @returns the result
 */
Query.prototype._find = async function _find() {
  const conds = this._conditions;
  const options = this._optionsForExec();

  if (this.$useProjection) {
    options.projection = this._fieldsForExec();
  } else {
    options.fields = this._fieldsForExec();
  }

  debug('find', this._collection.collectionName, conds, options);

  return this._collection.find(conds, options);
};

/**
 * Returns the query cursor
 *
 * #### Examples:
 *
 *     query.find().cursor();
 *     query.cursor({ name: 'Burning Lights' });
 *
 * @param {Object} [criteria] mongodb selector
 * @return {Object} cursor
 * @api public
 */

Query.prototype.cursor = function cursor(criteria) {
  if (this.op) {
    if (this.op !== 'find') {
      throw new TypeError('.cursor only support .find method');
    }
  } else {
    this.find(criteria);
  }

  const conds = this._conditions;
  const options = this._optionsForExec();

  if (this.$useProjection) {
    options.projection = this._fieldsForExec();
  } else {
    options.fields = this._fieldsForExec();
  }

  debug('findCursor', this._collection.collectionName, conds, options);

  return this._collection.findCursor(conds, options);
};
```

```
/**
 * Executes the query as a findOne() operation.
 *
 * #### Example:
 *
 *     query.findOne().where('name',  /^Burning/);
 *
 *     query.findOne({ name: /^Burning/ })
 *
 *     await query.findOne({ name: /^Burning/ }); // executes
 *
 * @param {Object|Query} [criteria] mongodb selector
 * @return {Query} this
 * @api public
 */

Query.prototype.findOne = function(criteria) {
  this.op = 'findOne';

  if (Query.canMerge(criteria)) {
    this.merge(criteria);
  }

  return this;
};

/**
 * Executes a `findOne` Query
 * @returns the results
 */
Query.prototype._findOne = async function _findOne() {
  const conds = this._conditions;
  const options = this._optionsForExec();

  if (this.$useProjection) {
    options.projection = this._fieldsForExec();
  } else {
    options.fields = this._fieldsForExec();
  }

  debug('findOne', this._collection.collectionName, conds, options);

  return this._collection.findOne(conds, options);
};

/**
 * Executes the query as a countDocuments() operation.
 *
 * #### Example:
 *
 *     query.countDocuments().where('color',  'black').exec();
 *
 *     query.countDocuments({ color: 'black' })
 *
 *     await query.countDocuments({ color: 'black' });
 *
 *     const count = await query.where('color', 'black').countDocuments();
 *     console.log('there are %d kittens', count);
 *
 * @param {Object} [filter] mongodb selector
 * @return {Query} this
 * @api public
 */

Query.prototype.countDocuments = function(filter) {
  this.op = 'countDocuments';
  this._validate();

  if (Query.canMerge(filter)) {
    this.merge(filter);
```

```
  }

  return this;
};

/**
 * Executes a `countDocuments` Query
 * @returns the results
 */
Query.prototype._countDocuments = async function _countDocuments() {
  const conds = this._conditions;
  const options = this._optionsForExec();

  debug('countDocuments', this._collection.collectionName, conds, options);

  return this._collection.countDocuments(conds, options);
};

/**
 * Executes the query as a estimatedDocumentCount() operation.
 *
 * #### Example:
 *
 *     query.estimatedDocumentCount();
 *
 *     const count = await query.estimatedDocumentCount();
 *     console.log('there are %d kittens', count);
 *
 * @return {Query} this
 * @api public
 */

Query.prototype.estimatedDocumentCount = function() {
  this.op = 'estimatedDocumentCount';
  this._validate();

  return this;
};

/**
 * Executes a `count` Query
 * @returns the results
 */
Query.prototype._estimatedDocumentCount = async function _estimatedDocumentCount() {
  const conds = this._conditions;
  const options = this._optionsForExec();
  debug('estimatedDocumentCount', this._collection.collectionName, conds, options);
  return this._collection.estimatedDocumentCount(conds, options);
};

/**
 * Declares or executes a distinct() operation.
 *
 * #### Example:
 *
 *     await distinct(criteria, field)
 *     distinct(criteria, field)
 *     await distinct(field)
 *     distinct(field)
 *     await distinct()
 *     distinct()
 *
 * @param {Object|Query} [criteria]
 * @param {String} [field]
 * @return {Query} this
 * @see mongodb http://www.mongodb.org/display/DOCS/Aggregation#Aggregation-
Distinct
 * @api public
 */
```

```
Query.prototype.distinct = function(criteria, field) {
  this.op = 'distinct';
  this._validate();

  if (!field && typeof criteria === 'string') {
    field = criteria;
    criteria = undefined;
  }

  if ('string' == typeof field) {
    this._distinctDoc = field;
  }

  if (Query.canMerge(criteria)) {
    this.merge(criteria);
  }

  return this;
};

/**
 * Executes a `distinct` Query
 * @returns the results
 */
Query.prototype._distinct = async function _distinct() {
  if (!this._distinctDoc) {
    throw new Error('No value for `distinct` has been declared');
  }

  const conds = this._conditions,
      options = this._optionsForExec();

  return this._collection.distinct(this._distinctDoc, conds, options);
};

/**
 * Declare and/or execute this query as an `updateMany()` operation. This function will
update _all_ documents that match
 * `criteria`, rather than just the first one.
 *
 * _All paths passed that are not $atomic operations will become $set ops._
 *
 * #### Example:
 *
 *     // Update every document whose `title` contains 'test'
 *     mquery().updateMany({ title: /test/ }, { year: 2017 })
 *
 * @param {Object} [criteria]
 * @param {Object} [doc] the update command
 * @param {Object} [options]
 * @return {Query} this
 * @api public
 */

Query.prototype.updateMany = function updateMany(criteria, doc, options) {
  return _update(this, 'updateMany', criteria, doc, options);
};

/**
 * Executes a `updateMany` Query
 * @returns the results
 */
Query.prototype._updateMany = async function() {
  return _updateExec(this, 'updateMany');
};

/**
 * Declare and/or execute this query as an `updateOne()` operation. This function will
_always_ update just one document,
 * regardless of the `multi` option.
```

```
 *
 * _All paths passed that are not $atomic operations will become $set ops._
 *
 * #### Example:
 *
 *     // Update the first document whose `title` contains 'test'
 *     mquery().updateMany({ title: /test/ }, { year: 2017 })
 *
 * @param {Object} [criteria]
 * @param {Object} [doc] the update command
 * @param {Object} [options]
 * @return {Query} this
 * @api public
 */

Query.prototype.updateOne = function updateOne(criteria, doc, options) {
  return _update(this, 'updateOne', criteria, doc, options);
};

/**
 * Executes a `updateOne` Query
 * @returns the results
 */
Query.prototype._updateOne = async function() {
  return _updateExec(this, 'updateOne');
};

/**
 * Declare and/or execute this query as an `replaceOne()` operation. Similar
 * to `updateOne()`, except `replaceOne()` is not allowed to use atomic
 * modifiers (`$set`, `$push`, etc.). Calling `replaceOne()` will always
 * replace the existing doc.
 *
 * #### Example:
 *
 *     // Replace the document with `_id` 1 with `{_id: 1, year: 2017 }`
 *     mquery().replaceOne({ _id: 1 }, { year: 2017 })
 *
 * @param {Object} [criteria]
 * @param {Object} [doc] the update command
 * @param {Object} [options]
 * @return {Query} this
 * @api public
 */

Query.prototype.replaceOne = function replaceOne(criteria, doc, options) {
  this.setOptions({ overwrite: true });
  return _update(this, 'replaceOne', criteria, doc, options);
};

/**
 * Executes a `replaceOne` Query
 * @returns the results
 */
Query.prototype._replaceOne = async function() {
  return _updateExec(this, 'replaceOne');
};

/*!
 * Internal helper for updateMany, updateOne
 */

function _update(query, op, criteria, doc, options) {
  query.op = op;

  if (Query.canMerge(criteria)) {
    query.merge(criteria);
  }

  if (doc) {
```

```
    query._mergeUpdate(doc);
  }

  if (utils.isObject(options)) {
    // { overwrite: true }
    query.setOptions(options);
  }

  return query;
}

/**
 * Helper for de-duplicating "update*" functions
 * @param {Query} query The Query Object (replacement for "this")
 * @param {String} op The Operation to be done
 * @returns the results
 */
async function _updateExec(query, op) {
  const options = query._optionsForExec();

  const criteria = query._conditions;
  const doc = query._updateForExec();

  debug(op, query._collection.collectionName, criteria, doc, options);
  return query._collection[op](criteria, doc, options);
}

/**
 * Declare and/or execute this query as a `deleteOne()` operation.
 *
 * #### Example:
 *
 *     await mquery(collection).deleteOne({ artist: 'Anne Murray' })
 *
 * @param {Object|Query} [criteria] mongodb selector
 * @return {Query} this
 * @api public
 */

Query.prototype.deleteOne = function(criteria) {
  this.op = 'deleteOne';

  if (Query.canMerge(criteria)) {
    this.merge(criteria);
  }

  return this;
};

/**
 * Executes a `deleteOne` Query
 * @returns the results
 */
Query.prototype._deleteOne = async function() {
  const options = this._optionsForExec();
  delete options.justOne;

  const conds = this._conditions;

  debug('deleteOne', this._collection.collectionName, conds, options);
  return this._collection.deleteOne(conds, options);
};

/**
 * Declare and/or execute this query as a `deleteMany()` operation. Always deletes
 * _every_ document that matches `criteria`.
 *
 * #### Example:
 *

 *     await mquery(collection).deleteMany({ artist: 'Anne Murray' })
```

```
 *
 * @param {Object|Query} [criteria] mongodb selector
 * @return {Query} this
 * @api public
 */

Query.prototype.deleteMany = function(criteria) {
  this.op = 'deleteMany';

  if (Query.canMerge(criteria)) {
    this.merge(criteria);
  }

  return this;
};

/**
 * Executes a `deleteMany` Query
 * @returns the results
 */
Query.prototype._deleteMany = async function() {
  const options = this._optionsForExec();
  delete options.justOne;

  const conds = this._conditions;

  return this._collection.deleteMany(conds, options);
};

/**
 * Issues a mongodb findOneAndUpdate command.
 *
 * Finds a matching document, updates it according to the `update` arg, passing any `options`,
and returns the found document (if any).
 *
 * #### Available options
 *
 * - `new`: bool - true to return the modified document rather than the original. defaults to true
 * - `upsert`: bool - creates the object if it doesn't exist. defaults to false.
 * - `sort`: if multiple docs are found by the conditions, sets the sort order to choose which doc
to update
 *
 * #### Examples:
 *
 *     await query.findOneAndUpdate(conditions, update, options) // executes
 *     query.findOneAndUpdate(conditions, update, options)  // returns Query
 *     await query.findOneAndUpdate(conditions, update) // executes
 *     query.findOneAndUpdate(conditions,  update)           // returns Query
 *     await query.findOneAndUpdate(update)              // returns Query
 *     query.findOneAndUpdate(update)                    // returns Query
 *     await query.findOneAndUpdate()                    // executes
 *     query.findOneAndUpdate()                          // returns Query
 *
 * @param {Object|Query} [query]
 * @param {Object} [doc]
 * @param {Object} [options]
 * @see mongodb http://www.mongodb.org/display/DOCS/findAndModify+Command
 * @return {Query} this
 * @api public
 */

Query.prototype.findOneAndUpdate = function(criteria, doc, options) {
  this.op = 'findOneAndUpdate';
  this._validate();

  if (Query.canMerge(criteria)) {
    this.merge(criteria);
  }

  // apply doc
```

```
  if (doc) {
    this._mergeUpdate(doc);
  }

  options && this.setOptions(options);

  return this;
};
/**
 * Executes a `findOneAndUpdate` Query
 * @returns the results
 */
Query.prototype._findOneAndUpdate = async function() {
  const conds = this._conditions;
  const update = this._updateForExec();
  const options = this._optionsForExec();

  return this._collection.findOneAndUpdate(conds, update, options);
};

/**
 * Issues a mongodb findOneAndReplace command.
 *
 * Finds a matching document, replaces it according to the `replacement` arg, passing any
`options`, and returns the found document (if any).
 *
 * #### Available options
 *
 * - `new`: bool - true to return the modified document rather than the original. defaults to true
 * - `upsert`: bool - creates the object if it doesn't exist. defaults to false.
 * - `sort`: if multiple docs are found by the conditions, sets the sort order to choose which doc
to update
 *
 * #### Examples:
 *
 *     await query.findOneAndReplace(conditions, replacement, options) // executes
 *     query.findOneAndReplace(conditions, replacement, options)  // returns Query
 *     await query.findOneAndReplace(conditions, replacement) // executes
 *     query.findOneAndReplace(conditions,  replacement)           // returns Query
 *     await query.findOneAndReplace(replacement)              // returns Query
 *     query.findOneAndReplace(replacement)                        // returns Query
 *     await query.findOneAndReplace()                       // executes
 *     query.findOneAndReplace()                             // returns Query
 *
 * @param {Object|Query} [query]
 * @param {Object} [replacement]
 * @param {Object} [options]
 * @see mongodb http://www.mongodb.org/display/DOCS/findAndModify+Command
 * @return {Query} this
 * @api public
 */

Query.prototype.findOneAndReplace = function(criteria, replacement, options) {
  this.op = 'findOneAndReplace';
  this._validate();

  if (Query.canMerge(criteria)) {
    this.merge(criteria);
  }

  // apply replacement
  if (replacement) {
    this._updateDoc = replacement;
    this.options = this.options || {};
    this.options.overwrite = true;
  }

  options && this.setOptions(options);
```

```
  return this;
};

/**
 * Executes a `findOneAndReplace` Query
 * @returns the results
 */
Query.prototype._findOneAndReplace = async function() {
  const conds = this._conditions;
  const replacement = this._updateForExec();
  const options = this._optionsForExec();

  debug('findOneAndReplace', this._collection.collectionName, conds, replacement, options);
  return this._collection.findOneAndReplace(conds, replacement, options);
};

/**
 * Issues a mongodb findOneAndDelete.
 *
 * Finds a matching document, removes it, returning the found document (if any).
 *
 * #### Available options
 *
 * - `sort`: if multiple docs are found by the conditions, sets the sort order to choose which doc
to update
 *
 * #### Examples:
 *
 *     await A.where().findOneAndDelete(conditions, options) // executes
 *     A.where().findOneAndDelete(conditions, options)  // return Query
 *     await A.where().findOneAndDelete(conditions) // executes
 *     A.where().findOneAndDelete(conditions) // returns Query
 *     await A.where().findOneAndDelete()   // executes
 *     A.where().findOneAndDelete()            // returns Query
 *
 * @param {Object} [filter]
 * @param {Object} [options]
 * @return {Query} this
 * @api public
 */

Query.prototype.findOneAndDelete = function(filter, options) {
  this.op = 'findOneAndDelete';
  this._validate();

  if (Query.canMerge(filter)) {
    this.merge(filter);
  }

  // apply options
  options && this.setOptions(options);

  return this;
};

/**
 * Executes a `findOneAndRemove` Query
 * @returns the results
 */
Query.prototype._findOneAndDelete = async function() {
  const options = this._optionsForExec();
  const conds = this._conditions;

  debug('findOneAndDelete', this._collection.collectionName, conds, options);
  return this._collection.findOneAndDelete(conds, options);
};

/**
 * Add trace function that gets called when the query is executed.
 * The function will be called with (method, queryInfo, query) and
```

```
 * should return a callback function which will be called
 * with (err, result, millis) when the query is complete.
 *
 * queryInfo is an object containing: {
 *   collectionName: <name of the collection>,
 *   conditions: <query criteria>,
 *   options: <comment, fields, readPreference, etc>,
 *   doc: [document to update, if applicable]
 * }
 *
 * NOTE: Does not trace stream queries.
 *
 * @param {Function} traceFunction
 * @return {Query} this
 * @api public
 */
Query.prototype.setTraceFunction = function(traceFunction) {
  this._traceFunction = traceFunction;
  return this;
};

/**
 * Executes the query
 *
 * #### Examples:
 *
 *     query.exec();
 *     await query.exec();
 *     query.exec('update');
 *     await query.exec('find');
 *
 * @param {String|Function} [operation]
 * @api public
 */

Query.prototype.exec = async function exec(op) {
  if (typeof op === 'string') {
    this.op = op;
  }

  assert.ok(this.op, 'Missing query type: (find, etc)');

  const fnName = '_' + this.op;

  // better error, because default would list it as "this[fnName] is not a function"
  if (typeof this[fnName] !== 'function') {
    throw new TypeError(`this[${fnName}] is not a function`);
  }

  return this[fnName]();
};

/**
 * Executes the query returning a `Promise` which will be
 * resolved with either the doc(s) or rejected with the error.
 *
 * @param {Function} [resolve]
 * @param {Function} [reject]
 * @return {Promise}
 * @api public
 */

Query.prototype.then = async function(res, rej) {
  return this.exec().then(res, rej);
};

/**
 * Returns a cursor for the given `find` query.
 *
 * @throws Error if operation is not a find
```

```
 * @returns {Cursor} MongoDB driver cursor
 */

Query.prototype.cursor = function() {
  if ('find' != this.op)
    throw new Error('cursor() is only available for find');

  const conds = this._conditions;

  const options = this._optionsForExec();
  if (this.$useProjection) {
    options.projection = this._fieldsForExec();
  } else {
    options.fields = this._fieldsForExec();
  }

  return this._collection.findCursor(conds, options);
};

/**
 * Determines if field selection has been made.
 *
 * @return {Boolean}
 * @api public
 */

Query.prototype.selected = function selected() {
  return !!(this._fields && Object.keys(this._fields).length > 0);
};

/**
 * Determines if inclusive field selection has been made.
 *
 *     query.selectedInclusively() // false
 *     query.select('name')
 *     query.selectedInclusively() // true
 *     query.selectedExlusively() // false
 *
 * @returns {Boolean}
 */

Query.prototype.selectedInclusively = function selectedInclusively() {
  if (!this._fields) return false;

  const keys = Object.keys(this._fields);
  if (0 === keys.
```

SYSTEM IMPLEMENTATION

4.2 Implementation Overview

The implementation of the Cara e-commerce application involves the integration of frontend, backend, and database components to create a fully functional system. The project follows a modular and layered approach to ensure scalability, maintainability, and efficient performance.

The implementation of Cara ensures a smooth interaction between frontend, backend, and database components. The modular design and use of modern technologies make the system efficient, secure, and scalable for real-world usage.

4.3 User Interface Screenshots

Figure 6.1: Login Page

The Login page provides a clean and secure authentication interface where registered users enter their email address and password to access TasteFlow. New users can navigate to the Sign Up page to register.

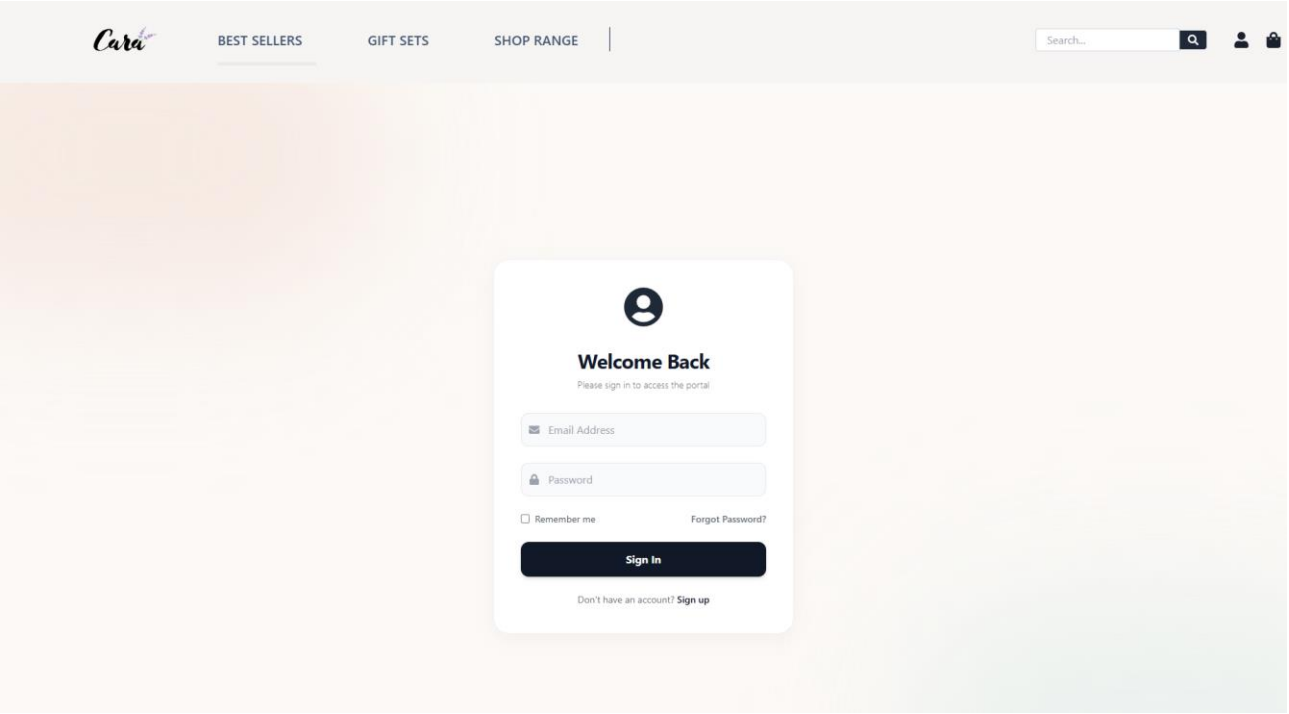

Figure 6.1: Login Page

Figure 6.2: Home Page

The Home Page features a prominent hero banner with the tagline 'Craving Something Delicious?' and provides quick access to the restaurant listing. After successful login, a 'Login Successful!' notification confirms authentication.

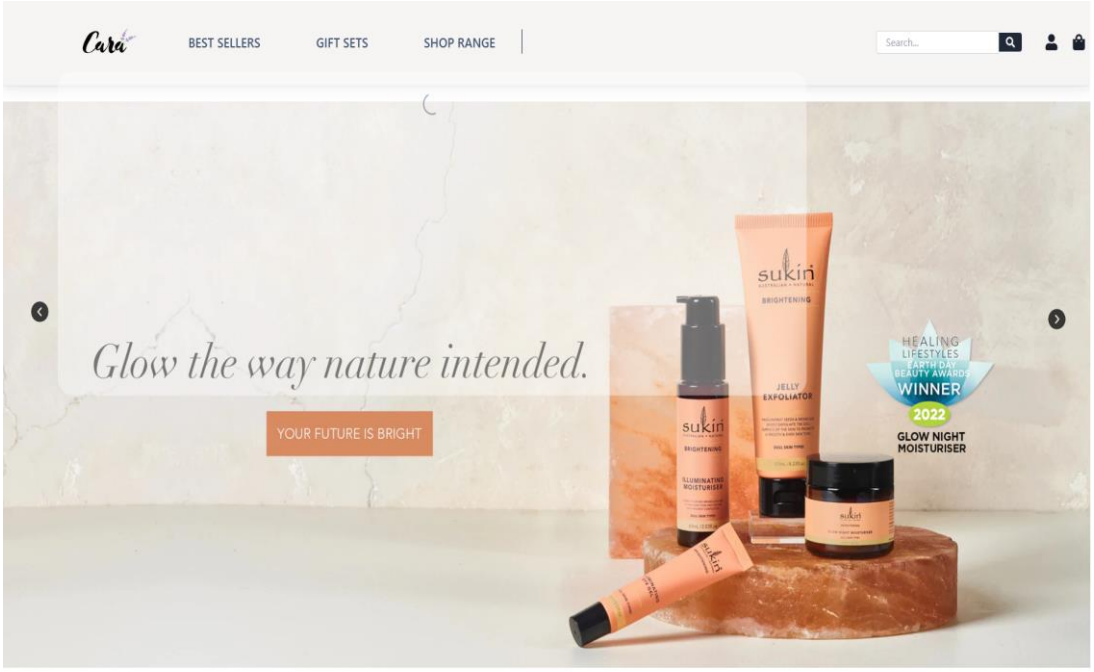

Figure 6.2: Home Page

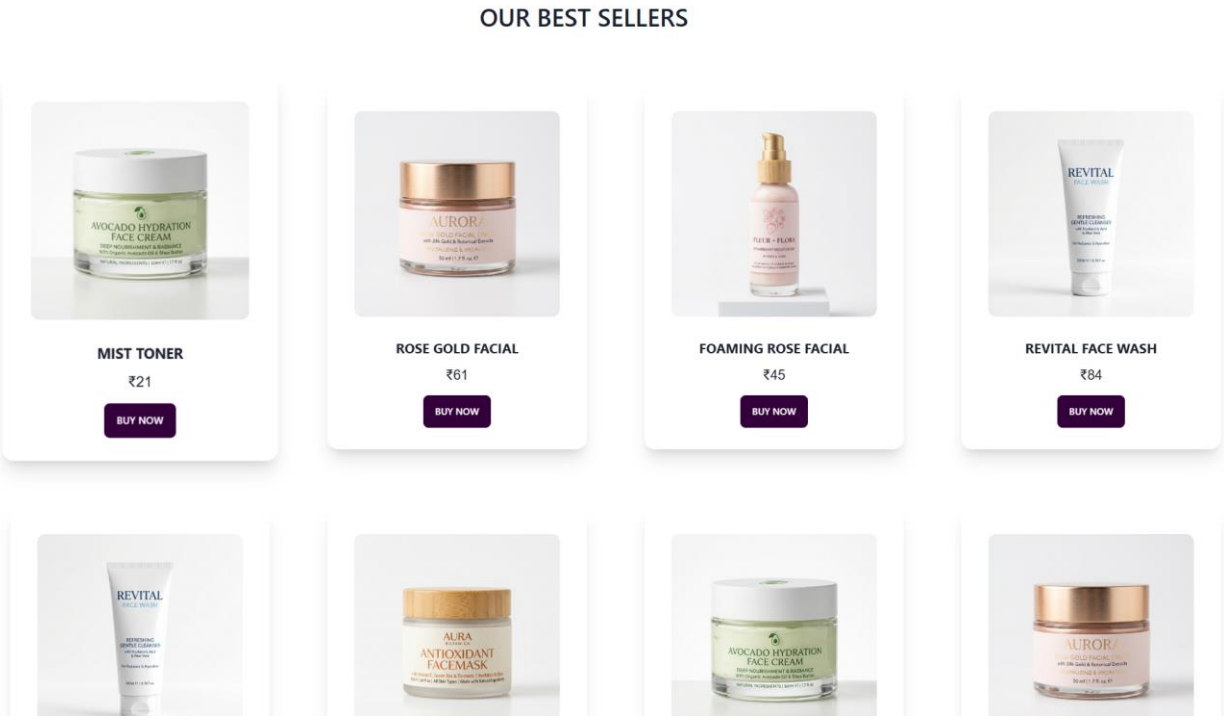

Figure 6.3: Features

The Featured Restaurants section displays restaurant cards with cover images, star ratings, names, descriptions, and address information. Users can select any restaurant to explore its full menu.

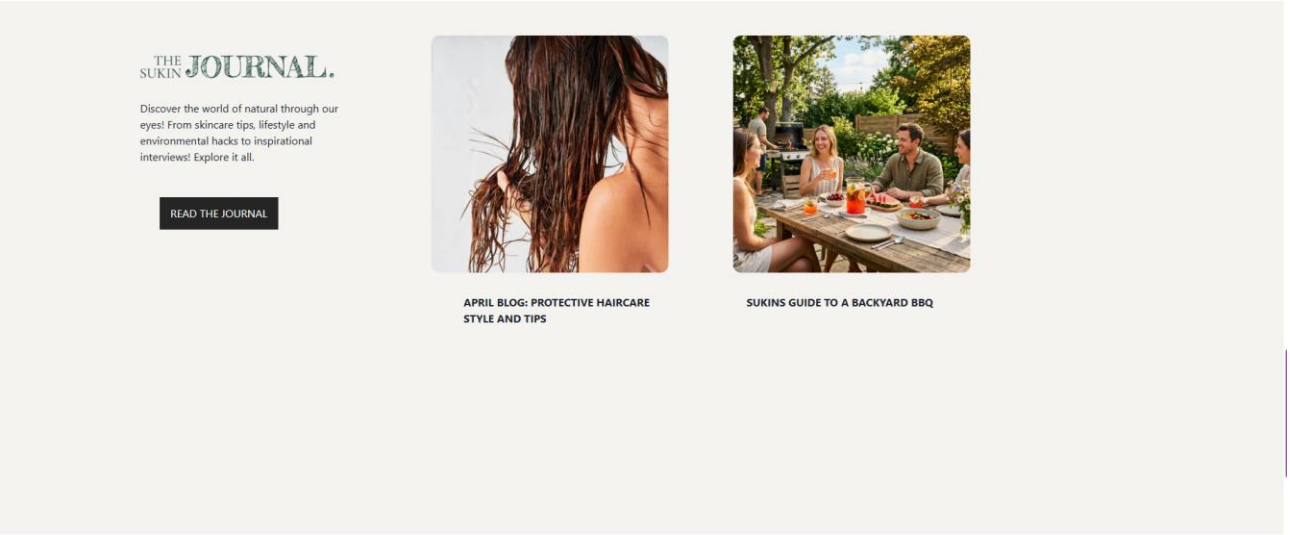

Figure 6.4: Features

Figure 6.4: All Products Page

The All Products page shows the complete listing including Neeraja (Indian, 4.7★), Karthik (Sushi & Ramen, 4.6★), and Sushmitha (Fusion Bowls, Tacos, Pizza, 4.8★). The page footer includes quick links and contact information.

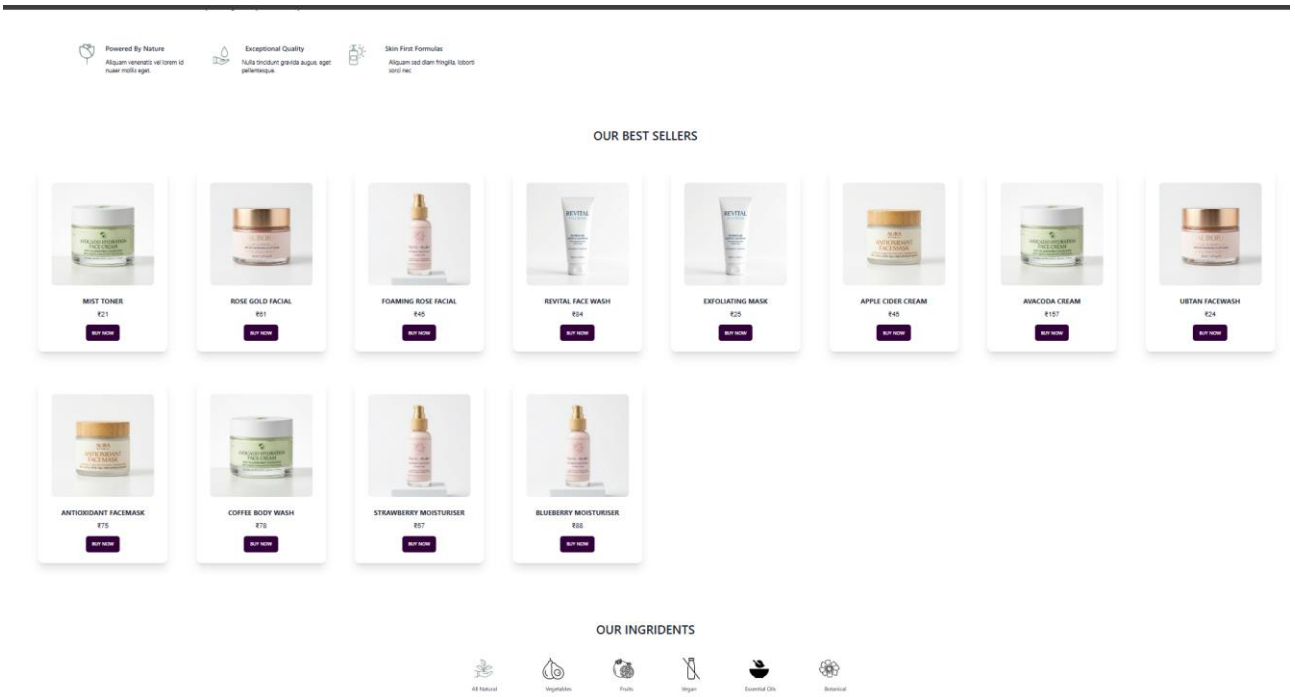

Figure 6.4: All Restaurants Page

Figure 6.5: Product Detail Page

The Restaurant Detail Page for 'FACE WASH ' displays a full-width cover image with the product name, cuisine description , star rating (4.6), and address. The complete menu is shown below the header.

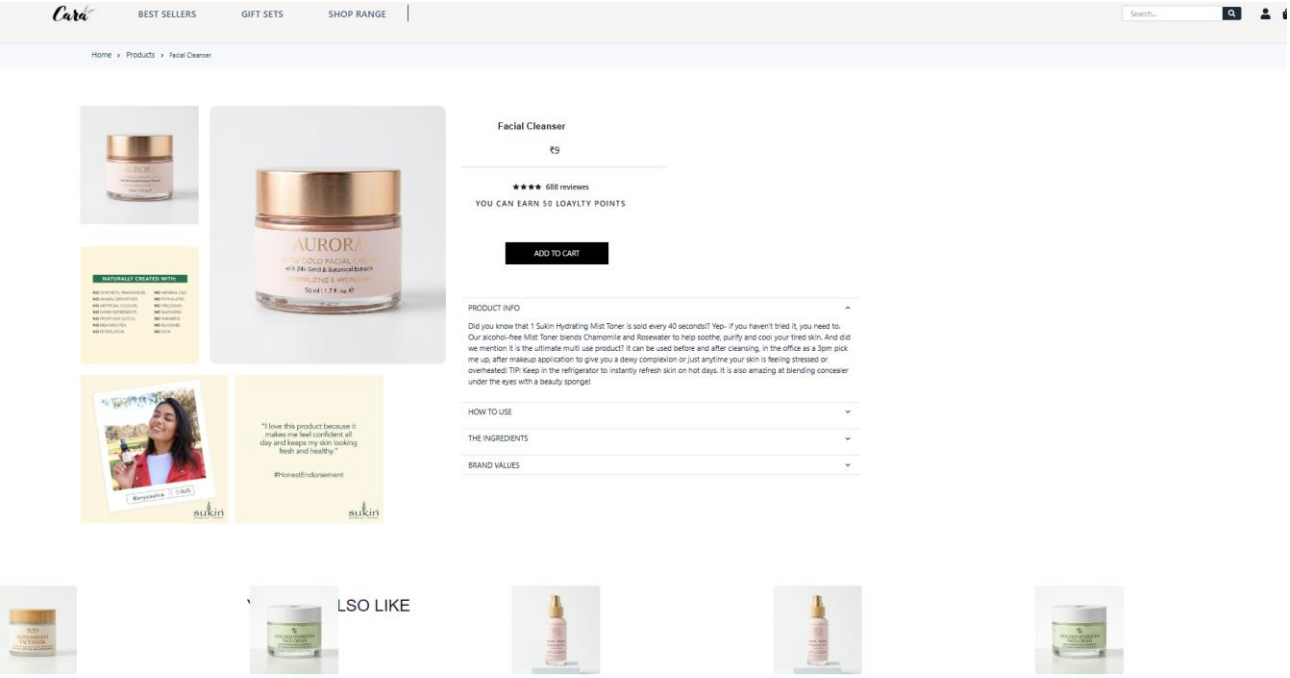

Figure 6.5: Product details

Figure 6.6: Search items

The Search items for Face wash displays Products items in a two-column grid. Each card shows the e-commerce image, name, category badge (e.g., 'Body wash'), price in INR, and a teal 'Add to Cart' button.

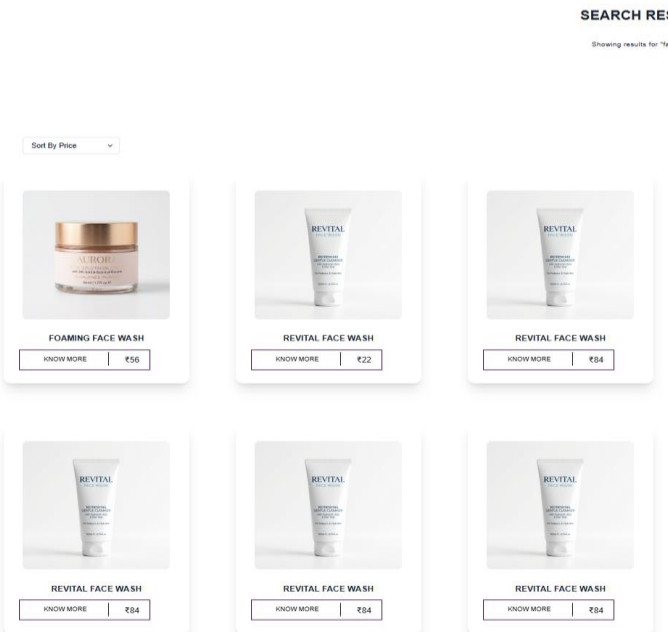

Figure 6.6: Search items

Figure 6.7: Check-out items

The **check-out items** feature in the cara e-commerce application is responsible for completing the purchase process. it allows users to review their selected products, enter delivery details, and confirm their order securely.

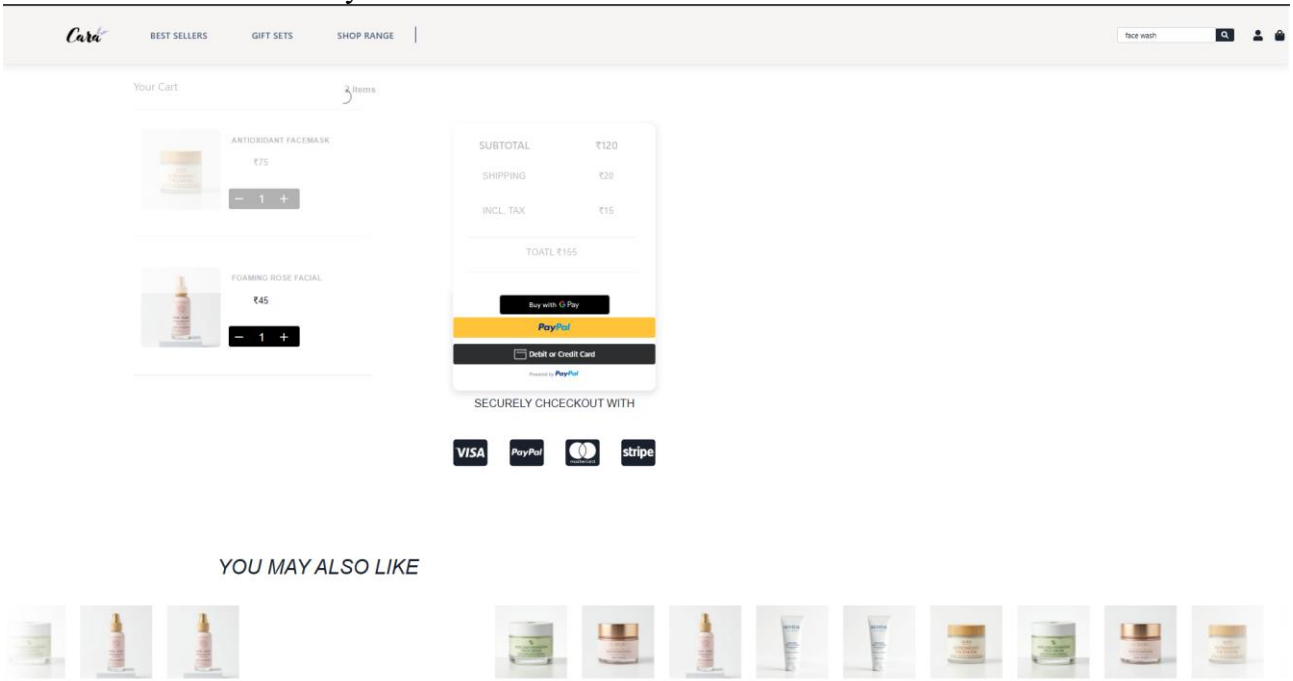

Figure 6.7: Check-out items

Figure 6.8: Admin portal

The **Admin Portal** in the Cara e-commerce application is a dedicated interface that allows administrators to manage the overall system. It provides complete control over products, users, and orders, ensuring smooth operation of the platform.

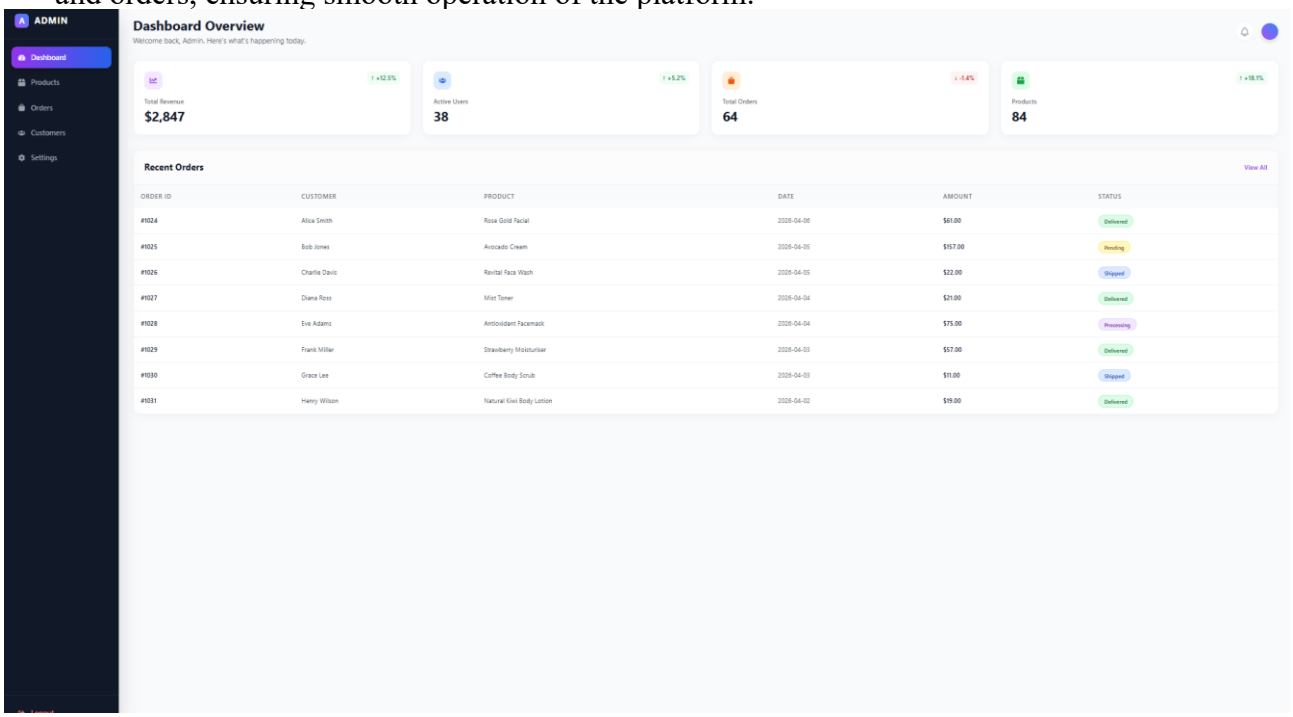

Figure 6.8: Admin Portal

5 CONCLUSION

The Cara e-commerce web application has been successfully designed and developed as a complete full-stack solution using modern technologies such as React.js, Node.js, Express.js, and MongoDB. The system effectively fulfills all the core requirements of an online shopping platform by providing features like user authentication, product browsing, cart management, order processing, and admin control.

The application offers a user-friendly and responsive interface, ensuring smooth navigation and an enhanced shopping experience. The backend system efficiently handles data processing, API communication, and database operations, while maintaining security through authentication mechanisms such as JWT and encrypted passwords.

The modular and scalable architecture of the system allows it to handle multiple users and large amounts of data, making it suitable for real-world applications. The integration of frontend, backend, and database components demonstrates the practical implementation of full-stack development concepts.

Thus, the project successfully achieves its objectives and showcases the effective use of modern web technologies in developing a reliable and efficient e-commerce system.

6 FUTURE ENHANCEMENTS

The Cara e-commerce application provides a strong foundation for an online shopping Platform. Future enhancements may include:

- Mobile Application Integration: Developing a cross-platform mobile application using React Native to cater to mobile-first users.
- AI-Powered Recommendations: Integrating machine learning algorithms to suggest restaurants and food items based on users' order histories and preferences.
- Real-time Order Tracking: Embedding a live order tracking interface using WebSockets to allow users to monitor delivery status in real time.
- Ratings and Reviews: Allowing users to rate restaurants and leave reviews for menu items, fostering a community-driven catalog.
- Payment Gateway Integration: Expanding the platform to support online payments by integrating third-party payment processors like Razorpay or Stripe.
- Multi-Language Support: Implementing internationalization (i18n) modules to make the platform accessible to a global, non-English speaking audience.
- Email Notifications: Integrating automated email services (e.g., Nodemailer) for account verification, password resets, and order confirmation alerts.
- OAuth Login: Permitting fast authentication via Google, GitHub, or Facebook OAuth providers to lower user onboarding friction.

7 REFERENCES

1. MongoDB Documentation. (n.d.). The MongoDB NoSQL Database. Retrieved from https://docs.mongodb.com/

2. Express.js Documentation. (n.d.). Fast, unopinionated, minimalist web framework for Node.js. Retrieved from https://expressjs.com/

3. React Official Documentation. (n.d.). A JavaScript library for building user interfaces. Retrieved from https://react.dev/

4. Node.js Documentation. (n.d.). Node.js v18.x API Reference. Retrieved from https://nodejs.org/docs/

5. JSON Web Tokens (JWT). (n.d.). Introduction to JSON Web Tokens. Retrieved from https://jwt.io/introduction

6. bcryptjs npm Package. (n.d.). Optimized bcrypt in JavaScript. Retrieved from https://www.npmjs.com/package/bcryptjs

7. Mongoose ODM Documentation. (n.d.). Elegant MongoDB object modeling for Node.js. Retrieved from https://mongoosejs.com/docs/

8. MDN Web Docs. (n.d.). JavaScript, HTML, and CSS References. Retrieved from https://developer.mozilla.org/